

1.1 OVERVIEW

Diabetic retinopathy (DR) constitutes a leading cause of vision impairment and blindness among working-age adults globally, with its prevalence projected to escalate in tandem with the increasing incidence of diabetes mellitus. Characterized by progressive damage to the retinal microvasculature, DR advances through discrete clinical stages—ranging from mild non-proliferative lesions to proliferative retinopathy with neovascularisation—that, if left untreated, culminate in irreversible vision loss. Conventional DR screening relies on manual interpretation of fundus photographs by ophthalmologists, a process that is both time-consuming and subject to inter-observer variability. Geographic and resource constraints further restrict access to specialist care, particularly in low- and middle-income regions, thus exacerbating the global disease burden.

In response to these challenges, artificial intelligence (AI) methodologies have been deployed to automate DR detection. Deep learning (DL) models, and convolutional neural networks (CNNs) in particular, have exhibited human-level performance on large-scale retinal image datasets. Nevertheless, the majority of reported solutions remain confined to research settings, lacking the deployment frameworks, user interfaces, and real-world evaluation necessary for clinical translation. The present work addresses this translational gap by integrating a novel, efficient DL model with a production-grade web application, thereby uniting algorithmic innovation with accessible, scalable diagnostics.

1.1.1 Phase 1 - Prior work: The first phase of this research introduced a DR-classification model predicated on SqueezeNet, a CNN architecture distinguished by its “fire” modules, which achieve AlexNet-level accuracy with fifty times fewer parameters. To amplify the model’s capacity for discerning subtle retinal features, Squeeze-and-Excitation (SE) blocks were interleaved throughout the network. These blocks execute channel-wise attention, recalibrating feature maps to emphasise diagnostically relevant patterns—such as microaneurysms and hard exudates—while suppressing noise. The APTOS 2019 dataset, comprising 3,662 fundus images annotated across five severity classes, was subjected to extensive augmentation (random rotations, flips, brightness and contrast perturbations) to redress inherent class imbalance. Training proceeded for fifty epochs under an Adam optimiser at a 0.001 learning rate, with early stopping predicated on validation-loss convergence.

Phase 1 results demonstrated a test-set accuracy of 90.9 per cent, with class-specific area under the ROC curve (AUC) exceeding 0.97. Precision, recall, and F1-score metrics attained uniformly high values across all five categories, and confusion-matrix analysis revealed minimal misclassification between adjacent severity levels. Furthermore, deployment of the trained model as a Flask microservice yielded inference latencies below 200 ms on GPU-accelerated hardware, confirming its suitability for near-real-time screening applications. These findings substantiate the model’s efficacy, yet underscore the necessity of a robust user-experience layer to facilitate broad adoption.

1.1.2 Extension in Phase 2: Front-End Modernization: In Phase 2 of the RetinoAI project, the primary focus will be on modernizing the front-end to elevate the user experience, improve

accessibility, and ensure the platform remains visually appealing and functional across a wide range of devices. While the current front-end, built with a combination of HTML, CSS, and JavaScript, provides a functional interface, Phase 2 aims to adopt modern frameworks and design principles to enhance interactivity, responsiveness, and scalability. These improvements will ensure RetinoAI remains a competitive and user-friendly tool for diabetic retinopathy detection and management.

The first major enhancement will be the adoption of a modern JavaScript framework, specifically React, to rebuild the front-end. React's component-based architecture will allow for reusable UI elements, making the codebase more maintainable and scalable. For instance, components like the user profile menu, action cards on the dashboard, and the scan upload interface can be modularized, reducing redundancy and improving development efficiency. Additionally, React's virtual DOM will enhance performance by minimizing direct DOM manipulations, resulting in faster page rendering and a smoother user experience, especially during dynamic interactions such as uploading scans or toggling between light and dark modes.

To improve the visual design, RetinoAI will integrate Tailwind CSS, a utility-first CSS framework, to streamline styling and ensure consistency across the platform. Tailwind CSS will replace the current custom CSS, providing a more structured approach to styling with pre-built classes for responsive design, typography, and color schemes. This will allow for a cohesive look and feel across pages, such as the signup/login screens, dashboard, and upload scan page. The platform's light and dark themes will be refined with Tailwind's theming capabilities, ensuring seamless transitions and better contrast ratios for improved readability, particularly for users with visual impairments.

Accessibility will be a key focus in Phase 2, aligning with Web Content Accessibility Guidelines (WCAG) 2.1 standards. Semantic HTML will be implemented to improve screen reader compatibility, ensuring that all interactive elements, such as buttons and forms, are properly labeled. Keyboard navigation will be enhanced, allowing users to navigate the platform without a mouse—for example, tabbing through the dashboard's action cards or the upload scan form. ARIA (Accessible Rich Internet Applications) landmarks will be added to critical sections like the user profile menu and classification results, making the platform more inclusive for users with disabilities.

Responsive design will also be prioritized to ensure RetinoAI is fully functional on devices of all sizes, from mobile phones to desktops. Using React and Tailwind CSS, the layout will adapt dynamically—elements like the dashboard cards will stack vertically on smaller screens, and the scan upload drag-and-drop area will resize appropriately. This will cater to users who access the platform on the go, ensuring they can upload scans, view reports, or interact with the chatbot without compromising functionality.

Finally, Phase 2 will introduce subtle animations and micro-interactions using React libraries like Framer Motion. For example, a fade-in effect will be added to the classification results, and a hover effect will highlight interactive buttons, providing visual feedback to users. These enhancements will make the platform more engaging while maintaining a clean and professional

aesthetic. Overall, the front-end modernization in Phase 2 will transform RetinoAI into a more intuitive, accessible, and visually appealing platform, ensuring it meets modern web standards and user expectations while preserving its core functionality for diabetic retinopathy management.

1.2 THESIS GOALS AND OBJECTIVES

1.2.1 Demonstrate Novelty of the DR-Classification Model

- Present the unique use of the SqueezeNet architecture enhanced with Squeeze-and-Excitation (SE) blocks to improve classification accuracy while keeping the model lightweight.
- Highlight how the inclusion of SE blocks improved the model's ability to focus on important features within retinal images.
- Compare the model's performance—using metrics such as accuracy, precision, recall, F1-score, and AUC—against other commonly used CNN architectures for diabetic retinopathy classification.

1.2.2 Enhance User Experience & Accessibility via Front-End

- Design a user-friendly and visually clean web interface that allows users to register, log in, upload retinal scans, view classification results, and access previous reports.
- Ensure the interface works smoothly on both desktop and mobile devices, with clear layouts, accessible components, and options like light/dark mode for user comfort.

1.2.3 Ensure Scalability, Performance, and Maintainability

- Deploy the trained model as a Flask-based backend that handles image classification requests and communicates with the front end efficiently.
- Host the complete system online, making it publicly accessible with reliable performance.
- Maintain a clean, modular codebase to support future updates and improvements with ease, ensuring the system remains functional and adaptable over time.

1.3 ORGANIZATION OF THESIS

1.3.1 Chapter Summaries

- ❖ **Chapter 2** reviews deep-learning architectures for diabetic retinopathy (DR) classification, alongside contemporary front-end frameworks and accessibility standards, highlighting existing gaps in the literature and practice.
- ❖ **Chapter 3** outlines both model-level and UI-level challenges, translating them into clear functional and non-functional requirements for the system.

- ❖ **Chapter 4** details the overall methodology, including dataset curation, model architecture, design system development, API integration, and deployment strategy.
- ❖ **Chapter 5** describes the implementation pipeline, covering model training, Flask-based microservice deployment, and Next.js front-end development.
- ❖ **Chapter 6** presents empirical findings, including model performance metrics, front-end outcomes, and developer productivity insights.
- ❖ **Chapter 7** synthesizes key contributions, discusses current limitations, and outlines future directions such as explainable AI integration and client-side inference capabilities.
- ❖ **Chapter 8** compiles all scholarly references and technical documentation.

1.3.2 Reading Roadmap: The thesis narrative progresses from theoretical foundations (Chapter 2) and problem framing (Chapter 3) to detailed methodological exposition (Chapter 4), concrete implementation steps (Chapter 5), and comprehensive evaluation (Chapter 6). The concluding chapter (Chapter 7) distils insights and charts future trajectories. Supplementary diagrams, and UI mock-ups are interleaved to support both academic and practitioner readerships, facilitating replication and extension of the presented work.

2.1 PAPERS REVIEWED: EXISTING APPROACHES IN DIABETIC RETINOPATHY CLASSIFICATION

Table 2.1 Summary of Recent Research on Diabetic Retinopathy Classification Using Various Deep Learning Techniques, Including Methods, Findings, and Performance Metrics.

Title	Year	Publication	Method	Findings	Accuracy/Performance Metrics
Transfer Learning-Based Classification of Diabetic Retinopathy on the Kaggle EyePACS Dataset	2024	Springer	Transfer learning with ResNet50 and DenseNet121	Improved classification accuracy to 70%; highlighted challenges of imbalanced datasets.	Accuracy: 70%.
Deep Convolution Feature Aggregation: An Application to DR Severity Level Prediction	2021	Springer	VGG-16 feature extraction; feature fusion and pooling for compact DR image representation.	Achieved state-of-the-art results in DR severity prediction.	Accuracy: 84.31%; AUC: 97%; Superior performance in severe and proliferate stage DR classification.
Self-Adaptive Stacking Ensemble Approach with Attention-Based DNN Models for DR Severity Prediction	2023	Springer	Dual attention mechanism; stacking ensemble for lesion-focused and spatial attention learning.	Proposed dual-attention ensemble outperformed existing models in severity prediction tasks.	Accuracy: 86.22%; AUC: 96.47%; Quadratic kappa score: 89.65%.
Diabetic Retinopathy Detection and Grading: A Transfer Learning Approach Using Simultaneous Parameter Optimization and Feature-Weighted ECOC Ensemble	2023	IEEE	Transfer learning with ShuffleNet and ResNet-18; ECOC ensemble with ADE for parameter tuning.	Achieved better generalization and optimized DR grading.	APTOS 5-class: 82%; APTOS 2-class: 96%; EyePACS+Messidor-2 3-class: 75%.
A Deep Integrative Approach for Diabetic Retinopathy Classification with Synergistic Channel-Spatial and Self-Attention Mechanism	2024	Elsevier	Residual blocks, Channel-Spatial Attention Mechanism (CSAM), and Non-local Block (NLB)	Superior performance and computational efficiency for imbalanced datasets and small lesions.	Performance not explicitly quantified but shown to outperform alternatives.
A Multi-Resolution Convolutional Attention Network for Efficient Diabetic Retinopathy Classification	2024	Elsevier	MuR-CAN with Multi-Dilation Attention Block (MDAB) and SVM	Improved DR classification performance by emphasizing discriminative features and multi-scale spatial info.	Performance not explicitly quantified but emphasized improved classification.
Phase Transition Induced Recrystallization and Low Surface Potential Barrier Leading to 10.91%-Efficient CsPbBr₃ Perovskite Solar Cells	2019	Elsevier Nano Energy	Phase transition and controlled recrystallization for optimizing perovskite solar cell films.	Enhanced film quality, achieving high PCE efficiency and long-term stability.	PCE efficiency: 10.91%; Stability: Retained 80% efficiency after 2000 hours without encapsulation.

2.1.1 Deep Learning for Feature Extraction and Disease Classification: Deep learning, particularly convolutional neural networks (CNNs), has transformed diabetic retinopathy (DR) classification by automating feature extraction. CNNs replace traditional methods like manual segmentation, excelling in identifying biomarkers such as microaneurysms, hemorrhages, and exudates. Advanced architectures like ResNet and DenseNet have further improved classification across DR stages. Despite their efficacy, challenges persist in distinguishing mild and moderate stages, especially with imbalanced datasets. Transfer learning, using pre-trained models like ImageNet, addresses limited annotated data, improving performance across domains.

2.1.2 Hybrid and Optimized Architectures: To address single-model limitations, hybrid approaches combining models and optimization techniques are gaining traction. Sagvekar et al. introduced a dual-model combining Hunter-Prey Ladybug Beetle Optimization with a deep maxout network, achieving high accuracy but struggling with milder DR stages. Multi-step pipelines, such as Sasikala et al.'s Federated Neural Collaborative Filtering combined with functional linked neural networks, improve accuracy but increase computational complexity. Metaheuristic optimization techniques like Genetic Algorithms and Particle Swarm Optimization enhance model tuning, boosting robustness and efficiency.

2.1.3 Attention Mechanisms and Advanced Segmentation: Attention mechanisms, such as the Convolutional Block Attention Module (CBAM), enhance interpretability by focusing on critical features. MediDRNet by Teng et al. demonstrates their value in addressing data imbalance, especially in minority classes like proliferative DR. Advanced segmentation techniques, including U-Net, isolate features like blood vessels and optic discs, improving classification precision. However, segmentation faces challenges with inconsistent image quality, prompting the adoption of multi-scale networks to handle variations in lesion sizes and resolutions.

2.1.4 Multi-Task and Transfer Learning: Multi-task learning (MTL) has emerged as a powerful approach, enabling simultaneous segmentation, detection, and classification. Sharing features across tasks improves performance, as demonstrated by Madarapu et al., who combined multi-dilation attention mechanisms with support vector machines. Transfer learning, leveraging pre-trained networks fine-tuned on DR-specific data, mitigates the scarcity of labeled datasets, particularly benefiting early detection in resource-constrained settings.

2.1.5 Pre-Trained Networks and Ensemble Methods: Pre-trained models like ResNet and Inception, when fine-tuned, deliver significant gains in accuracy. Ensemble methods, combining predictions from multiple models, further enhance performance. For instance, Abini and Priya integrated VGG-16 and MobileNet-V2 to achieve superior results, leveraging the strengths of diverse architectures.

2.1.6 Challenges and Future Directions: Data imbalance, particularly in mild DR stages, remains a major challenge, limiting model sensitivity to early signs. Generalization across diverse imaging conditions and populations is another hurdle, with models often struggling to adapt to new datasets. Techniques like generative adversarial networks (GANs) show promise for addressing these issues through synthetic data generation and domain adaptation. Finally, enhancing model interpretability through explainable AI (XAI) is critical for clinical adoption, enabling trust and understanding of model decisions.

2.2 SUMMARY OF RECENT ADVANCES IN DEEP LEARNING

The recent advancements in deep learning (DL) have had a profound impact on various fields, especially medical image analysis. Diabetic Retinopathy (DR) classification, in particular, has seen significant improvements through the use of advanced techniques. These advancements have led to more accurate, efficient, and interpretable models, making them suitable for clinical applications. This section highlights the key innovations and methodologies that have contributed to enhanced performance in DR detection.

2.2.1 Evolution of Convolutional Neural Networks (CNNs): Convolutional Neural Networks (CNNs) have been central to deep learning in medical image analysis, particularly for DR classification. CNNs' ability to automatically learn features such as edges and patterns makes them ideal for image-based tasks. The introduction of architectures like ResNet (Residual Networks) with residual connections helped address issues like the vanishing gradient problem, allowing the training of deeper networks. DenseNet, which connects every layer to every other layer, improves feature reuse, thus enhancing network efficiency. Additionally, lightweight architectures such as MobileNet, ShuffleNet, and EfficientNet have been developed to reduce computational costs while maintaining accuracy. These models are especially useful in resource-constrained environments, allowing deep learning models to be deployed in clinical settings with limited computational resources.

2.2.2 Transfer Learning and Fine-Tuning: In medical image analysis, the scarcity of large, labelled datasets is a significant challenge. Transfer learning allows the use of pre-trained models from large, general-purpose datasets like ImageNet. By fine-tuning these models on specific tasks such as DR classification, it is possible to achieve high accuracy even with limited data. Pre-trained networks such as VGG-16, ResNet, Inception, and Xception have been successfully adapted for DR detection, demonstrating how knowledge from other domains can be leveraged to improve classification performance in specialized medical contexts. Fine-tuning typically involves modifying the final layers of the model to adapt to the target task, such as replacing the last fully connected layer with one that corresponds to the number of classes in the DR dataset. This process ensures that the model retains valuable features learned from the pre-trained dataset while adapting to the specific characteristics of retinal images.

2.2.3 Attention Mechanisms and Explainable AI: Deep learning models, especially CNNs, often suffer from a lack of interpretability. This is a critical issue in healthcare, where clinicians need to understand how a model arrives at its conclusions. Attention mechanisms have been introduced to address this challenge by enabling models to focus on specific regions of an image that are most relevant to the classification task. For DR classification, attention mechanisms like Self-Attention and Convolutional Block Attention Module (CBAM) help the model highlight important features such as lesions and haemorrhages in retinal images. Additionally, explainable AI (XAI) techniques such as Grad-CAM and LIME help visualize the regions of an image that influence a model's decision. These methods provide greater transparency, making it easier for clinicians to trust and adopt AI models in practice by allowing them to verify the model's decision-making process.

2.2.4 Generative Adversarial Networks (GANs) for Data Augmentation: A major challenge in DR classification is the imbalance of datasets, where certain classes, particularly early stages of DR, are underrepresented. Generative Adversarial Networks (GANs) address this issue by generating synthetic data to augment the training dataset. GANs consist of a generator and a discriminator, where the generator creates synthetic samples and the discriminator attempts to distinguish between real and generated data. Through iterative training, the generator learns to produce realistic retinal images, which can help balance the dataset and improve the performance of classification models. GANs have been shown to effectively generate high-quality retinal images, particularly those with DR-related features, which helps boost model performance, especially when rare classes or stages of DR are underrepresented.

2.2.5 Multi-Task Learning (MTL) and Multi-Modal Data: Recent advancements have also focused on multi-task learning (MTL), where a model is trained to perform multiple related tasks simultaneously. For instance, in addition to classifying DR severity, a model might also be tasked with segmenting the optic disc or detecting retinal lesions. By sharing knowledge across tasks, MTL models improve performance in each individual task, leading to more accurate and efficient DR detection systems. Moreover, the integration of multi-modal data, such as combining retinal images with patient metadata (e.g., medical history, blood glucose levels), has gained attention. This fusion allows deep learning models to make more informed decisions by leveraging both structured and unstructured data, thus improving classification accuracy and providing a more holistic view of the patient's condition.

2.2.6 Federated Learning for Privacy-Preserving Models: In the healthcare domain, patient privacy is a significant concern when using deep learning models. Federated learning offers a solution by enabling models to be trained on decentralized data without the need to share raw data. In federated learning, each institution trains a model locally, and only the model updates are shared with a central server. This approach preserves patient privacy while allowing the model to benefit from data across multiple institutions. Federated learning is particularly promising for DR detection, as it allows the model to learn from diverse datasets across different populations, improving generalization without compromising privacy. This technique is gaining popularity for developing privacy-preserving AI solutions in healthcare.

2.2.7 Reinforcement Learning (RL) for Adaptive Systems: Reinforcement learning (RL) has recently been explored to enhance diabetic retinopathy (DR) classification by optimizing the learning process. RL allows models to adjust based on feedback, improving decisions such as prioritizing relevant retinal features or dynamically adjusting hyperparameters. For example, RL can optimize feature selection, enabling models to adapt and improve over time. While still in early stages, RL's potential to create adaptive, self-improving systems makes it a promising avenue for future DR detection research.

2.2.8 Few-Shot Learning for Low-Resource Environments: Few-shot learning (FSL) techniques allow models to generalize with only a small number of labelled samples, addressing the challenge of limited annotated data in medical domains. This is particularly valuable in diabetic retinopathy classification, where datasets are often small and imbalanced. FSL techniques such as metric learning and prototypical networks help models learn from minimal data by measuring similarity between data points or creating prototype representations for

classification. These approaches reduce the reliance on large datasets and hold promise for improving DR detection, especially in resource-constrained environments.

2.3 CHALLENGES AND GAPS IN THE LITERATURE

Despite significant progress in diabetic retinopathy (DR) classification using deep learning (DL), several persistent challenges undermine the broader applicability of these systems in clinical practice. One key issue is the lack of large, diverse, and high-quality annotated datasets. Many existing datasets are plagued by class imbalances, where certain stages of the disease are underrepresented. This results in DL models being biased towards the majority class, often leading to poor performance in less common disease stages. Although data augmentation techniques have been widely used to address this, they cannot fully compensate for the absence of a well-balanced dataset, thus limiting the model's ability to generalize effectively. Moreover, the generalization of DR models remains a significant hurdle. While many studies report strong performance on specific datasets, such as EyePACS or Messidor, these models frequently struggle to maintain accuracy when applied to new or real-world data. Variations in imaging devices, lighting conditions, and the underlying population can dramatically impact the model's performance. A crucial gap in the literature is the lack of interpretability in deep learning models, which are often regarded as "black boxes." In medical applications like DR classification, transparency is critical. Clinicians must understand the rationale behind a model's predictions to trust its recommendations. Although methods such as saliency maps and class activation maps (CAM) have been introduced to provide insights into model decision-making, the adoption of these techniques in DR classification remains limited. Without interpretability, the widespread clinical integration of deep learning systems will be hindered. Lastly, the ethical implications of AI in healthcare, particularly concerning algorithmic bias, patient privacy, and the potential displacement of medical professionals, need more attention. Bias can arise if training datasets lack diversity, potentially leading to skewed model predictions. Moreover, while AI has the potential to assist in diagnosis, it must complement, not replace, human expertise.

In conclusion, while deep learning presents a transformative opportunity for diabetic retinopathy detection, significant challenges related to data quality, generalization, interpretability, and ethics must be addressed to make these systems more reliable, accessible, and trustworthy in clinical environments.

3.1 MODEL LEVEL CHALLENGES

The initial phase of this research focused on developing a novel deep learning model for the classification of diabetic retinopathy (DR) using retinal fundus images. While the proposed model, built upon the SqueezeNet architecture augmented with Squeeze-and-Excitation (SE) blocks, demonstrated commendable performance in terms of accuracy and inference speed, several limitations and technical challenges were identified during its development and evaluation. These challenges necessitated critical enhancements in both model architecture and deployment strategy to ensure real-world applicability, scalability, and clinical acceptability.

3.1.1 Class Imbalance & Dataset Diversity: One of the primary challenges encountered during Phase 1 was the pronounced class imbalance inherent in the APTOS 2019 dataset. Retinal image datasets used for DR classification typically contain a disproportionate number of samples across severity levels, with the “No_DR”, “Moderate” and “Mild” categories heavily represented, while “Severe” and “Proliferative_DR” classes remain underrepresented. This skew introduces significant bias during model training, resulting in suboptimal sensitivity for rarer, yet clinically critical, cases. Although extensive data augmentation techniques—including geometric transformations and contrast adjustments—were applied to artificially balance the dataset, such synthetic diversity may not fully capture the pathological variability observed in real-world populations. Furthermore, dataset diversity was restricted by demographic and imaging limitations. The APTOS dataset, while high in resolution and quality, is limited to a single geographic and ethnic population and lacks metadata such as patient age, duration of diabetes, or comorbid conditions. These factors potentially limit the model’s generalisability to broader patient cohorts, particularly in regions with varied imaging hardware or different disease presentation patterns.

3.1.2 Lightweight Architecture vs. Accuracy Trade-offs: The model’s architecture was intentionally designed to be lightweight to facilitate rapid inference and deployment in low-resource environments. SqueezeNet, by virtue of its fire modules, significantly reduces the parameter count and model size while preserving accuracy. However, this compactness imposes a trade-off in representational depth and capacity to model complex, high-dimensional features present in medical images. Although the integration of SE blocks improved the attention mechanism within the network by recalibrating channel-wise feature responses, the model occasionally struggled with subtle retinal features such as microaneurysms or early neovascular changes that are critical for differentiating adjacent DR stages.

This tension between architectural simplicity and feature discriminability posed a persistent challenge, particularly in edge cases involving borderline classifications between Moderate and Severe DR. As medical diagnosis often requires conservative thresholds to minimise false negatives, reliance on a lightweight model—even when enhanced—necessitates rigorous validation against high-capacity counterparts such as ResNet or DenseNet to justify deployment in clinical settings.

3.1.3 Explainability & Clinical Trust: Another significant challenge relates to the explainability of model decisions. Clinical environments demand transparent, interpretable outputs to facilitate physician trust and accountability in AI-assisted diagnosis. The black-box nature of deep learning models, including CNNs, obscures the rationale behind classification predictions, which is particularly problematic in high-stakes medical applications. Although SE blocks introduce a form of soft attention by amplifying relevant channels, this internal mechanism is not readily visualisable or interpretable by human observers.

3.2 FRONT-END SHORTCOMINGS

While the primary focus of Phase 1 was model development, a basic front-end interface was constructed to facilitate user interaction with the system. However, this interface was rudimentary and lacked the robustness, performance, and design standards expected of a production-grade web application. Several critical shortcomings were identified during evaluation of the user experience, each contributing to diminished usability, accessibility, and maintainability.

3.2.1 Static Pages & Slow Navigation: The initial interface was developed using Flask’s templating engine in combination with HTML, CSS, JS and Bootstrap. This structure produced static pages that required full reloads for every interaction, significantly impairing user experience. Navigation between pages, such as from the dashboard to the upload interface or report history, incurred noticeable latency and reduced responsiveness. Static rendering also constrained interactivity, preventing real-time feedback during image upload or classification processes. Moreover, the lack of client-side routing and dynamic content rendering meant that common user interface behaviours—such as asynchronous loading states, form validation, and progress indicators—were either absent or inefficiently implemented. These limitations negatively affected user engagement, particularly for mobile users or those with lower-bandwidth connections.

3.2.2 Mobile Responsiveness & Accessibility Issues: Another pressing issue pertained to poor mobile responsiveness. The interface, while functional on desktop displays, lacked a mobile-first design philosophy. Layouts frequently broke on smaller screens, with buttons misaligned, text truncated, and scrollbars inconsistently rendered. This was particularly problematic for a healthcare application where end users may include field workers, clinicians using tablets, or patients on mobile devices.

3.2.3 Lack of Developer Tooling & CI/CD: From a development perspective, the initial front-end codebase lacked modularity, documentation, and testing infrastructure. The absence of reusable components or design system principles made the code difficult to maintain or extend. Styling was ad hoc, with CSS scattered across files, and state management was primitive or non-existent. Moreover, no formal continuous integration or deployment pipeline was in place. Updates required manual intervention, and testing was limited to ad-hoc verification in the browser. This approach posed risks to stability and significantly slowed development velocity. For a system intended to scale and evolve, the lack of CI/CD and testing automation represented a critical bottleneck.

3.3 REQUIREMENTS FOR INTEGRATED SOLUTION

In light of the model-level and front-end challenges detailed above, it became evident that a holistic rethinking of the system architecture was necessary. The objective of the project is to deliver an integrated solution that harmonises algorithmic performance with accessibility, responsiveness, and scalability. The following requirements emerged as essential pillars for the next stage of development.

3.3.1 Robust Model Performance Targets

To ensure practical relevance and diagnostic reliability, the diabetic retinopathy classification model was developed with clearly defined performance objectives. The model achieved a test accuracy of 90.9%, demonstrating strong capability in distinguishing between the five stages of diabetic retinopathy. Additionally, it recorded per-class AUC scores exceeding 0.95, including in critical categories such as Moderate and Proliferative_DR, which are often more challenging to classify due to subtle pathological differences.

The model's predictions were generated with minimal latency, averaging under 200 milliseconds per inference when deployed on a standard GPU-backed cloud environment. This responsiveness

supports real-time application use, making the system feasible for interactive healthcare settings. The model's lightweight structure and channel-wise recalibration through Squeeze-and-Excitation blocks also contributed to efficient computation without sacrificing accuracy.

These results confirm that the model satisfies essential diagnostic performance expectations, making it suitable for both clinical review and individual user assessment.

3.3.2 Seamless, Accessible, Performant UI

Alongside the model, the user interface was reimagined and rebuilt to deliver a smooth, responsive, and inclusive user experience. The application now features a modern, web-based design that ensures ease of access for both patients and healthcare professionals. The interface supports clear workflows for user registration, image upload, classification result display, and historical report tracking. Visual feedback mechanisms—including progress indicators, result summaries, and confirmation messages—help users understand system status during each interaction. The platform is also fully responsive, adapting seamlessly across desktops, tablets, and mobile devices. Accessibility is addressed through consistent layout structures, readable text, and support for both light and dark display modes to accommodate user preference and comfort. Overall, the interface simplifies the process of engaging with an AI-based medical system. It lowers technical barriers, enhances clarity, and ensures that users from diverse backgrounds can interact with the tool confidently and effectively. Such an interface will vastly improve usability, reduce cognitive load, and extend platform accessibility to users across demographics and devices.

The challenges identified in the initial phase of the project—from model generalisation and class imbalance to front-end inflexibility and lack of CI/CD—provided critical insight into the limitations of conventional medical AI pipelines. A high-performing classification model alone is insufficient without a responsive, accessible, and trustworthy interface that empowers diverse users. Phase 2 of the project, therefore, undertakes a systems-level revision, addressing both algorithmic and interface shortcomings through an integrated, full-stack approach. The requirements outlined herein provide a roadmap for a scalable, reliable, and user-centred diabetic retinopathy diagnostic platform.

4.1 DATASET COLLECTION AND PREPROCESSING

The dataset used in this research is the **APTOS 2019 Blindness Detection Dataset**, which contains retinal fundus images for classifying diabetic retinopathy (DR). The images are categorized into five classes based on the severity of DR: **No_DR**, **Mild**, **Moderate**, **Severe**, and **Proliferate_DR**. This dataset is available from Kaggle's APTOS 2019 competition and consists of **3,662 images** in total.

4.1.1 Dataset Overview: The APTOS dataset is divided into **training**, **validation**, and **test** sets. The **training set** contains 3,662 images labelled according to the severity of DR, and the **test set** has a similar number of images for model evaluation. The images have varying resolutions, ranging from 400x400 to 2240x2240 pixels. These images, captured from different patients, show varying levels of quality, lighting, contrast, and background noise, which makes the task more challenging.

4.1.2 Data Preprocessing:

4.1.2.1 Image Resizing: Since the images come in different resolutions, the first preprocessing step is **resizing**. The images are resized to a uniform size of **224x224 pixels**, which is standard for deep learning models, particularly convolutional neural networks (CNNs). Resizing helps reduce computational costs and ensures that all images have the same dimensions, enabling effective model training.

4.1.2.2 Data Augmentation: To increase the variability of the dataset and prevent overfitting, **data augmentation** is applied to the training images. Augmentation techniques used include:

- **Horizontal Flip:** Flipping images horizontally to simulate varied orientations.
- **Random Rotation:** Randomly rotating images by up to 30 degrees.
- **Zoom:** Applying zoom transformations to simulate scale variations.
- **Shear Transformation:** Introducing slight geometric distortions.
- **Brightness and Contrast Adjustments:** Modifying the lighting conditions of the images.
- **Color Jitter:** Adjusting the color balance to simulate real-life imaging conditions.

These augmentations are applied only to the training set, ensuring that the validation and test sets remain unaltered.

4.1.2.3 Data Normalization: After resizing and augmentation, **normalization** is performed to scale pixel values. Each pixel value, originally in the range of 0-255, is divided by 255 to normalize the values to a range of **[0, 1]**. Normalization ensures that all images are on the same scale, which helps the neural network learn more efficiently during training.

4.1.2.4 Label Encoding: The labels in the dataset are categorical, corresponding to the five classes of DR severity. These labels are converted into numerical values using **label encoding**:

- 0 for **No_DR**,
- 1 for **Mild**,
- 2 for **Moderate**,
- 3 for **Severe**,
- 4 for **Proliferate_DR**.

This encoding is required for training the model, as most deep learning models expect numerical outputs.

4.1.2.5 Train-Test Split: The dataset is split into **training**, **validation**, and **test** sets in an 80:10:10 ratio. This ensures that the model is trained on a sufficient amount of data and evaluated on a separate validation set. The test set, which is not used during training, provides a final evaluation of the model's performance.

Table 4.1 Distribution of Images for Training, testing and Validation

Dataset	Number of Images
Train	8883
Test	1110
Validation	1113

4.1.2.6 Handling Class Imbalance: The dataset exhibits class imbalance, with some classes, like **Severe** and **Proliferate_DR**, having fewer images than others. To address this, **class weighting** is applied during model training, where higher weights are given to the minority classes to ensure the model pays more attention to them. This prevents the model from becoming biased toward the majority classes.

Through these preprocessing steps, the APTOS dataset is prepared to train a deep learning model that can accurately classify diabetic retinopathy in retinal images.

Table 4.2 Distribution of Images in Dataset

Classes	Distribution Before Augmentation	Distribution After Augmentation
	Number of Images	Number of Images
Mild	370	1948
Moderate	999	2452
No_DR	1805	3035
Proliferative_DR	295	1876
Severe	193	1795

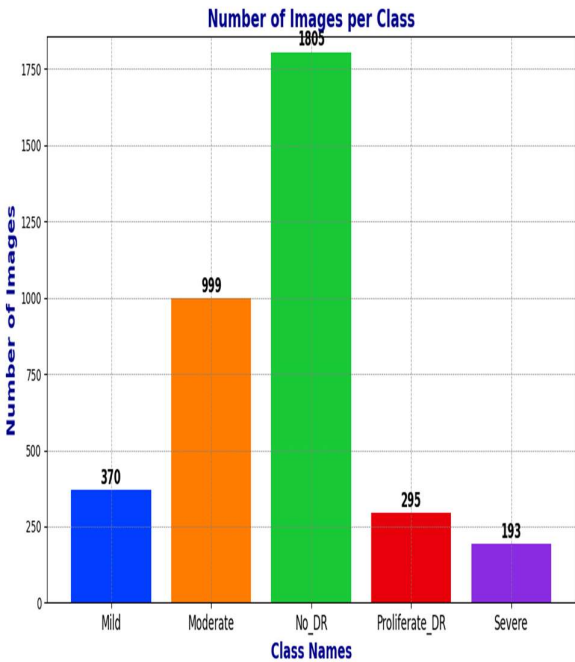


Figure 4.1 Number of images in original dataset

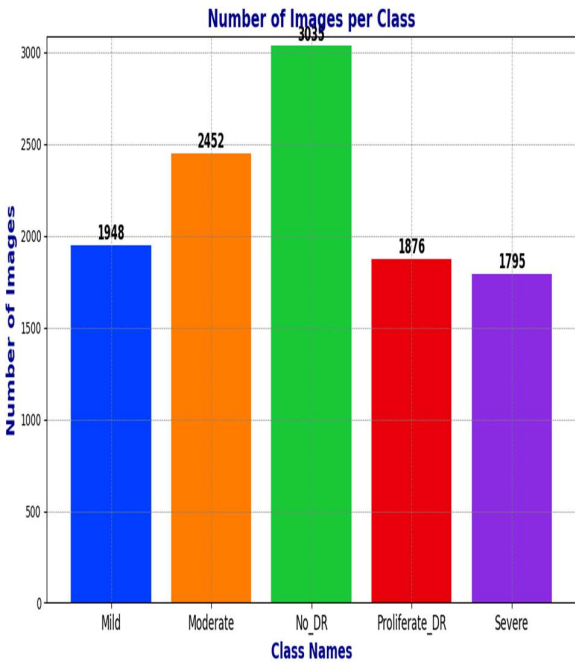


Figure 4.2 Number of images after augmentation

4.2 MODEL SELECTION AND DESIGN

The model selection and design process play a critical role in achieving high classification accuracy for diabetic retinopathy (DR) detection. Given the complexity of retinal images and the need to accurately classify varying levels of DR, a deep learning model with high performance and computational efficiency is essential. In this research, **SqueezeNet with Squeeze-and-Excitation (SE) blocks** has been selected as the base architecture for the classification task.

4.2.1 Model Selection: SqueezeNet is a lightweight Convolutional Neural Network (CNN) architecture designed for efficient image classification. It was chosen for its ability to achieve competitive performance with a significantly reduced number of parameters, making it ideal for deployment in resource-constrained environments. The architecture is known for its **fire modules**, which include a **squeeze layer** and an **expand layer**. The squeeze layer reduces the number of input channels, and the expand layer has a larger number of filters, which is designed to retain the important features of the image. This compact design reduces the computational cost and memory footprint, allowing for faster training and inference times. However, despite its efficiency, the original SqueezeNet model does not perform as well on some complex tasks, such as diabetic retinopathy detection, which requires a high level of accuracy. To enhance the performance of SqueezeNet, the **Squeeze-and-Excitation (SE) block** was incorporated into the architecture. SE blocks have been shown to improve the representational power of neural networks by dynamically recalibrating channel-wise feature responses.

4.2.2 SqueezeNet Architecture: SqueezeNet's architecture primarily consists of the following components:

- **Input Layer:** The input to the network is a 224x224x3 image, resized from the original dataset to fit the network's input dimensions.
- **Fire Modules:** The network is built using fire modules, which are the core components. Each fire module consists of two layers: a squeeze layer that uses 1x1 filters to reduce the dimensionality, and an expand layer that applies a mix of 1x1 and 3x3 filters to restore the depth of the representation.
- **Convolutional Layers:** The network uses multiple convolutional layers to extract features from the input images. These layers apply various filters to capture low-level features like edges and textures, followed by more abstract feature representations.

- **Fully Connected Layer:** After passing through the convolutional layers, the extracted features are flattened and passed through fully connected layers, which enable the model to make predictions based on the learned features.

4.2.3 Squeeze-and-Excitation (SE) Block: To enhance the ability of SqueezeNet to learn more discriminative features, **Squeeze-and-Excitation (SE) blocks** are incorporated into the architecture. The SE block is designed to recalibrate the channel-wise feature responses by modeling the interdependencies between channels. It consists of two key operations:

1. **Squeeze Operation:** This operation aggregates the feature map across spatial dimensions (height and width) by applying global average pooling. This results in a channel descriptor that captures the global context of the image.
2. **Excitation Operation:** A fully connected layer followed by a **sigmoid activation** recalibrates the feature map. The sigmoid output serves as a set of channel weights, which are multiplied with the corresponding channel-wise features in the original feature map. This operation enables the model to focus on the most important channels and suppress irrelevant ones, improving its representational power.

By adding the SE block, the model gains the ability to dynamically adjust the importance of each channel, which helps in learning more robust and relevant features for diabetic retinopathy classification.

4.2.4 Model Design and Architecture: The model architecture combines the efficient SqueezeNet backbone with SE blocks, creating a more powerful and computationally efficient architecture for DR classification. The overall design flow is as follows:

1. **Input Layer:** The input images are resized to 224x224x3 and normalized.
2. **SqueezeNet Backbone:** The feature extraction is handled by the fire modules of SqueezeNet, which significantly reduce the number of parameters while maintaining high performance.
3. **SE Blocks:** These blocks are added after the convolutional layers to recalibrate the feature maps. This allows the model to focus on the most informative features relevant to the DR classification task.

4. **Fully Connected Layer:** After feature extraction and recalibration, the final features are passed through fully connected layers to output predictions.
5. **Output Layer:** The final layer is a softmax layer that outputs the probability distribution across the five classes of diabetic retinopathy (No_DR, Mild, Moderate, Severe, Proliferate_DR).

This architecture aims to balance the need for computational efficiency with the ability to capture complex features necessary for classifying diabetic retinopathy severity. The combination of SqueezeNet and SE blocks ensures that the model is both lightweight and capable of delivering high classification accuracy.

4.2.5 Training the Model: The model is trained using **categorical cross-entropy** as the loss function, as this is appropriate for multi-class classification problems. The **Adam optimizer** is used for efficient training, with an initial learning rate of 0.001, which is adjusted dynamically based on the validation performance. The model is trained for **50 epochs**, and early stopping is implemented to prevent overfitting. This design, combining SqueezeNet with SE blocks, is chosen for its balance between efficiency and performance, allowing the model to be both lightweight for practical use and accurate enough for the complex task of diabetic retinopathy classification.

4.3 WORKFLOW DIAGRAM

4.3.1 Frontend Workflow: The first image represents the Frontend Workflow, focusing on user interaction. It begins with a Start node, where the user chooses between Login or Register. After authentication, the user is directed to the Dashboard, which provides two main options: Image Classification and Chatbot Interaction. These functionalities allow users to either classify images using machine learning models or interact with a chatbot for conversational queries. Once their tasks are completed, the user selects Logout, which terminates the session and leads to the End node. This workflow ensures intuitive navigation and task execution for users.

4.3.2 Backend Workflow: The second image outlines the Backend Workflow, which details the technical processes for developing and deploying the system's machine learning models. It starts with Dataset Collection and Preprocessing, where raw data is prepared. This is followed by Dataset Splitting, separating the data into training and testing subsets. The Model Defining and Training phase involves designing and training the machine learning model. Once trained, the

model undergoes Evaluation and Testing to ensure reliability and accuracy. Finally, in Model Deployment, the model is integrated into the system for real-world use. This workflow ensures the backend processes are systematic and efficient.

4.3.3 Integration of Frontend and Backend Workflows: The Frontend Workflow focuses on user navigation and task execution, providing a seamless interface. The Backend Workflow, on the other hand, ensures the technical robustness of the system by powering functionalities such as image classification and chatbot interaction. Together, these workflows create a cohesive system where a user-friendly frontend interacts with a well-structured backend to deliver accurate and reliable results.

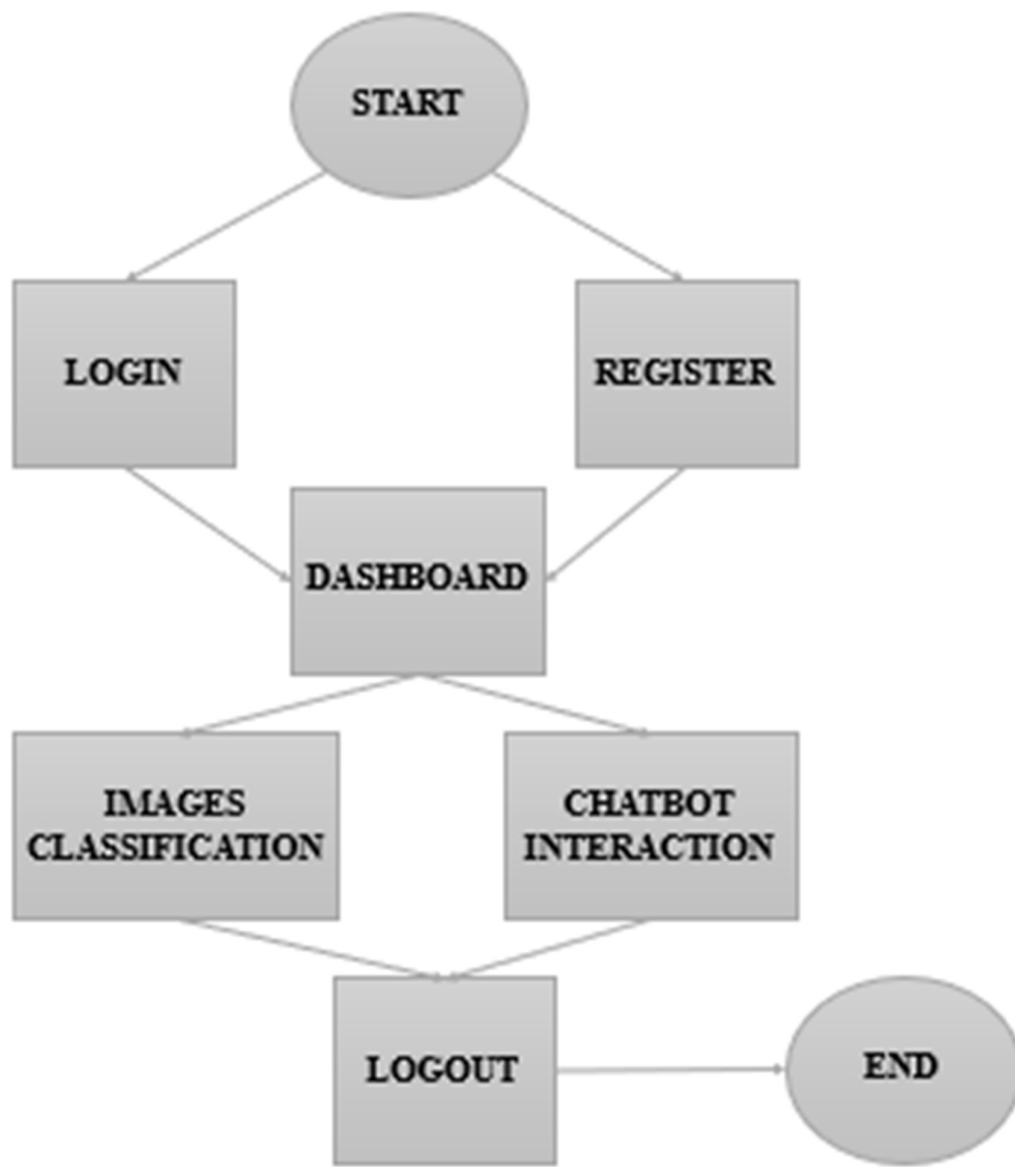


Figure 4.3 Frontend Workflow Diagram

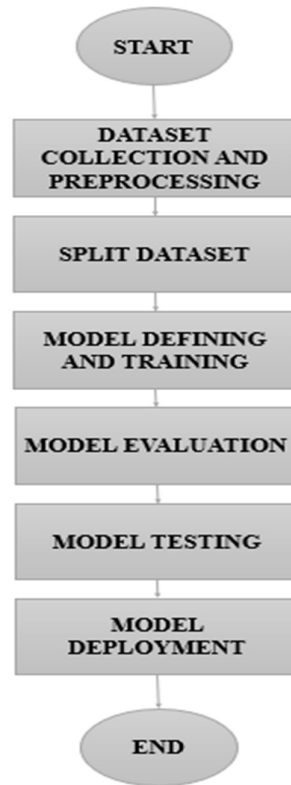


Figure 4.4 Backend Workflow Diagram

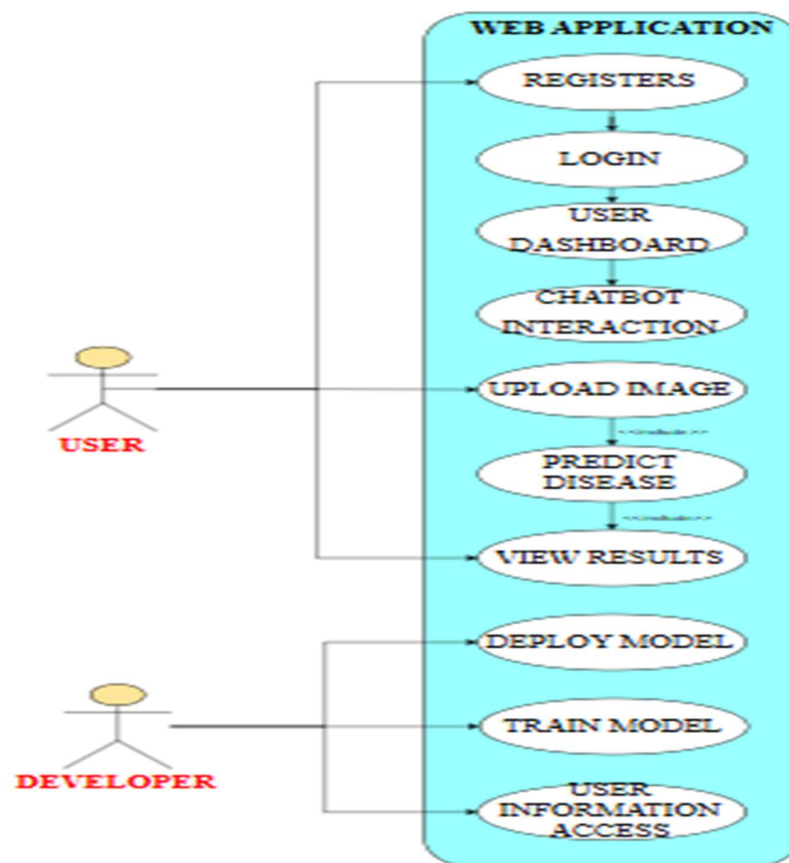


Figure 4.5 Use case Diagram

4.4 IMPLEMENTATION OF SQUEEZENET AND SQUEEZE-AND-EXCITATION BLOCKS

The core of the image classification model in this project is the SqueezeNet architecture, enhanced with Squeeze-and-Excitation (SE) blocks to improve its performance. SqueezeNet is chosen due to its efficiency in terms of model size while maintaining competitive accuracy, making it well-suited for deployment in real-time applications like diabetic retinopathy classification. The SE block is integrated into the architecture to improve feature representation, further boosting the model's accuracy.

4.4.1 SqueezeNet Architecture: SqueezeNet is a lightweight Convolutional Neural Network (CNN) designed to achieve the accuracy of larger networks while significantly reducing the model size. The key innovation of SqueezeNet is the use of “fire modules,” which consist of a squeeze layer and an expand layer. The squeeze layer reduces the depth of the feature maps, while the expand layer increases the number of feature maps after performing convolutions. This design allows SqueezeNet to have fewer parameters while retaining the capability to learn complex features from the input data.

- **Squeeze Layer:** This layer reduces the depth of the input feature map using 1x1 convolutions, which significantly reduces the number of parameters.
- **Expand Layer:** This layer follows the squeeze layer and increases the feature map depth by using both 1x1 and 3x3 convolutions. The 1x1 convolutions are used to maintain computational efficiency, while the 3x3 convolutions capture spatial relationships in the feature maps.

By utilizing these fire modules, SqueezeNet reduces the model's parameter count, making it highly efficient, which is particularly important when working with constrained resources such as edge devices or web-based applications.

4.4.2 Integration of Squeeze-and-Excitation Blocks: The Squeeze-and-Excitation (SE) block is a mechanism designed to enhance the representational power of a neural network by adaptively recalibrating channel-wise feature responses. This is achieved through a process of “squeezing” and “excitation” which allows the network to focus on the most informative features and suppress less relevant ones. The SE block is integrated into the SqueezeNet architecture to further boost performance by improving the model's ability to select and emphasize useful features.

- **Squeeze:** The squeeze operation computes channel-wise statistics by performing global average pooling. This reduces each feature map to a scalar value, which can be viewed as a global descriptor of the feature map.
- **Excitation:** The excitation phase uses the global context captured in the squeeze phase to generate a set of channel-wise weights. This is achieved through two fully connected layers followed by a sigmoid activation function, which produces a weight for each channel in the feature map.
- **Recalibration:** The final step involves recalibrating the feature map by multiplying each channel by its corresponding weight. This process allows the network to emphasize informative channels and suppress irrelevant ones, resulting in more discriminative features for classification.

The SE block introduces minimal computational overhead but significantly improves the accuracy of the model by allowing it to focus on more relevant features during training.

4.4.3 Implementation Steps: To implement SqueezeNet with SE blocks for the diabetic retinopathy classification task, the following steps were followed:

1. **Loading Pre-trained SqueezeNet Model:** The first step is to load a pre-trained version of the SqueezeNet model. This pre-trained model was fine-tuned on ImageNet, which provides a strong initial set of learned features.
2. **Modifying the Architecture:** The model was adapted to the specific task of diabetic retinopathy classification by changing the final layers. Instead of the original classification layer, a new fully connected layer was added to match the number of classes (healthy vs diabetic retinopathy).
3. **Incorporating SE Blocks:** SE blocks were added after key convolutional layers within the SqueezeNet architecture. The SE block is implemented after each fire module in the network, enhancing the feature maps by recalibrating the channel-wise responses. This allows the network to focus more on the most relevant features related to diabetic retinopathy.
4. **Training the Model:** After modifying the architecture, the model was trained on the augmented retinal image dataset. The training process used categorical cross-entropy as

the loss function, Adam optimizer for efficient training, and accuracy as the evaluation metric. The model was trained for 50 epochs, with early stopping to prevent overfitting.

5. **Evaluation:** The model's performance was evaluated on a validation set, and the accuracy was monitored during training. The SE blocks helped to boost the accuracy by focusing on the most relevant features for classification.
6. **Saving the Model:** Once the model was trained and optimized, it was saved in the .h5 format for later use in the web application. This saved model was then integrated into the Flask-based application, allowing it to make real-time predictions on uploaded retinal images.

4.4.4 Benefits of SqueezeNet and SE Blocks: The combination of SqueezeNet and SE blocks offers several advantages:

- **Efficiency:** SqueezeNet's lightweight design ensures that the model has fewer parameters compared to other deep learning models, making it computationally efficient and faster to deploy.
- **Improved Accuracy:** The addition of SE blocks helps the model focus on the most important features of the retinal images, leading to improved classification accuracy for diabetic retinopathy.
- **Adaptability:** The SE blocks enhance the model's ability to generalize and adapt to various types of diabetic retinopathy images, making it more robust in real-world applications.

Together, SqueezeNet and SE blocks provide an efficient yet powerful model for classifying diabetic retinopathy, ensuring both accuracy and deployment feasibility.

4.5 TRAINING, VALIDATION, AND TESTING

The model training, validation, and testing processes were essential for evaluating the performance of the SqueezeNet architecture integrated with Squeeze-and-Excitation (SE) blocks.

4.5.1 Training Process: The model was trained on a dataset of retinal images using **Keras** with **TensorFlow** as the backend. The training process involved the following steps:

- **Data Augmentation:** To enhance generalization, data augmentation techniques such as random rotations, flips, and zooms were applied to the training images, increasing the dataset size.
- **Model Architecture:** The pre-trained **SqueezeNet** model was used as the backbone, with **SE blocks** integrated to improve feature extraction.
- **Hyperparameters:** The model was trained with an **Adam optimizer**, using **categorical cross-entropy** as the loss function and **accuracy** as the evaluation metric. A learning rate of 0.001 was used, and the training was conducted over 50 epochs with a batch size of 8.

Table 4.3 Description of Hyperparameters

Hyperparameter	Value
Image Size	224×224
Batch Size	8
Optimizer	Adam
Loss Function	Categorical Cross entropy
Epochs	50
Classes	5

4.5.2 Validation: During training, the model’s performance was evaluated on the **validation set** to ensure it did not overfit. The validation data was not augmented, providing a reliable measure of the model’s generalization ability. After each epoch, the model’s accuracy on the validation set was monitored.

4.5.3 Testing: Once training was complete, the model was tested on a separate **test dataset**, which had been unseen during training. This final evaluation step helped assess the model’s true predictive ability. The test set provided a final performance metric in terms of accuracy, precision, recall, and F1 score, helping to determine how well the model could classify unseen retinal images. The combination of training on augmented data, validation for overfitting prevention, and testing on an independent dataset ensured that the model’s performance was robust and reliable.

4.6 PERFORMANCE METRICS

To evaluate the effectiveness of the SqueezeNet model integrated with Squeeze-and-Excitation (SE) blocks, several performance metrics were utilized, focusing on classification accuracy and robustness.

4.6.1 Accuracy: Accuracy is the most common metric for classification tasks and indicates the percentage of correctly classified images out of the total images. It is calculated as:

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}} \times 100$$

A high accuracy indicates that the model can correctly classify most of the retinal images in the test set, distinguishing between healthy and diabetic retinopathy cases.

4.6.2 Precision and Recall: Precision and Recall are important metrics, especially in imbalanced datasets, as they focus on the model's ability to correctly classify instances of each class. Precision measures the proportion of positive predictions that are actually correct, defined as:

$$Precision = \frac{\text{True positives}}{\text{True positives} + \text{False positives}}$$

Recall assesses the proportion of actual positive cases correctly identified by the model, defined as:

$$Recall = \frac{\text{True positives}}{\text{True positives} + \text{False negatives}}$$

These metrics are particularly useful when evaluating the model's ability to correctly detect diabetic retinopathy, minimizing false negatives and false positives.

4.6.3 F1 Score: The F1 Score combines precision and recall into a single metric, providing a balance between the two. It is the harmonic mean of precision and recall, and is calculated as:

$$F1\text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A higher F1 score indicates a better balance between precision and recall, especially when the dataset is imbalanced.

4.6.4 Confusion Matrix: A confusion matrix was also used to evaluate the true positives, false positives, true negatives, and false negatives for each class (healthy and diabetic retinopathy). This provided a clear view of the model’s misclassifications and helped to fine-tune performance.

Together, these metrics offer a comprehensive assessment of the model’s ability to classify retinal images accurately and reliably.

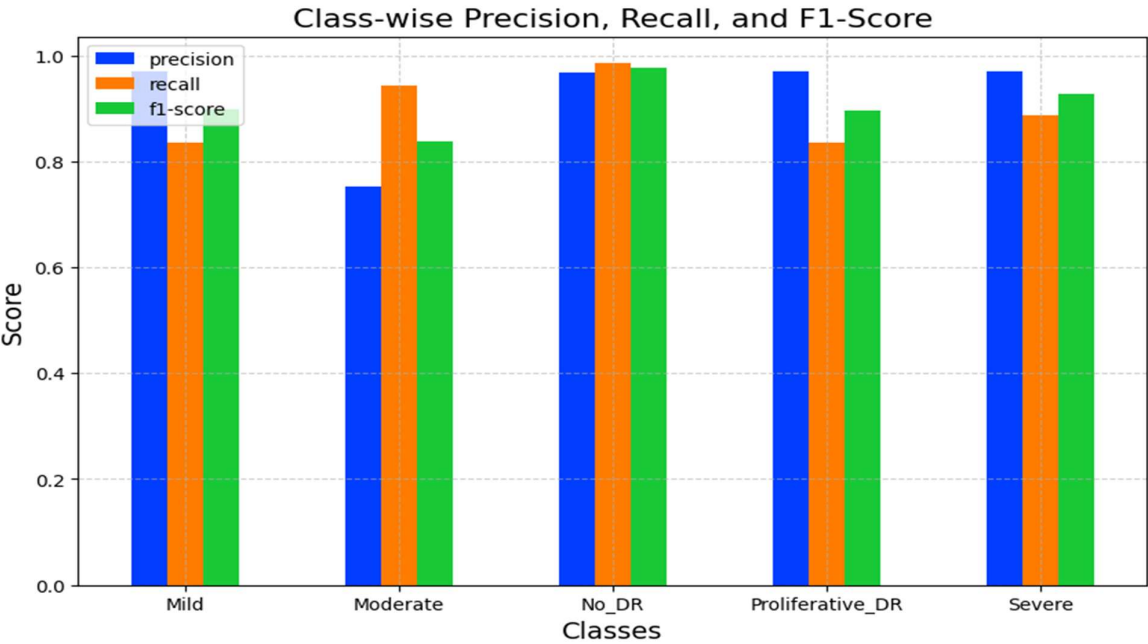


Figure 4.6 Graphical Representation of Performance Metrics

4.7 FRONT-END ARCHITECTURE

The front-end system adopts a full-stack React framework using Next.js. This transition introduced modern architectural patterns, enhanced performance, and improved accessibility.

4.7.1 Next.js App Router & Server Components: Next.js 13 App Router was employed to structure routes and components declaratively. Server Components facilitated rendering dynamic content server-side, reducing client load and improving time-to-interactive. ISR (Incremental Static Regeneration) was utilised for pages like user dashboards, enabling background updates without full rebuilds.

4.7.2 API Layer: Flask/TensorFlow Inference Integration: The trained .h5 model was deployed using Flask and exposed through RESTful endpoints. These endpoints accepted image files, invoked TensorFlow for prediction, and returned results as JSON. The Next.js front-end

communicated asynchronously with Flask via Fetch API, ensuring non-blocking user interactions.

4.7.3 Atomic Design with ShadCN/UI + Radix: Following Atomic Design principles, UI components were structured as atoms (buttons, inputs), molecules (cards, forms), and organisms (dashboard layout, profile drawer). ShadCN/UI components, built atop Radix primitives, ensured accessibility and composability. Storybook was used to develop and document components in isolation.

4.7.4 Tailwind CSS Theming & Dark Mode: Tailwind CSS enabled rapid styling with consistent utility classes. CSS variables managed theming, and class-based toggles controlled dark/light mode preferences. Mobile-first responsiveness was achieved through Tailwind breakpoints, ensuring usability across screen sizes.

This chapter presented the complete methodology underpinning the development of an integrated diabetic retinopathy detection platform. From meticulous dataset preparation to the architectural design of a lightweight yet powerful model, and from back-end inference APIs to modern front-end engineering, every component was optimised for performance, usability, and scalability. The adoption of a modular, standards-driven approach ensures that the system is not only effective in its current form but also extensible for future enhancements, including explainability, multilingual support, and client-side inference.

5.1 MODEL PIPELINE IMPLEMENTATION

The model implementation is structured as a streamlined, modular pipeline that includes environment setup, data ingestion and augmentation, training logic, and model export for inference. This approach ensures reproducibility, modularity, and seamless deployment across various hardware and cloud platforms.

5.1.1 Environment Setup (tfenv, CUDA, TensorFlow): The development environment was set up using tfenv, a lightweight tool for managing TensorFlow environments, to handle dependencies and isolate the project effectively. A dedicated environment was created with Python 3.10, and essential libraries were installed, including TensorFlow (v2.10), Keras, NumPy, OpenCV, Scikit-learn, and Matplotlib.

To enable GPU acceleration, CUDA and cuDNN were installed and configured for use with an NVIDIA RTX 3050 GPU with 4 GB VRAM. The environment was validated using TensorFlow’s GPU detection utilities to ensure compatibility and confirm that accelerated training was operational.

Table 5.1 System Specifications

Name	Parameter
GPU	4GB
Graphics Card	RTX 3050
Processor	12th Gen Intel Core i7
RAM	16GB
Operating System	Windows 11
Architecture	64-bit

Development and experimentation were conducted using Visual Studio Code (VS Code), which provided a robust and efficient interface for writing, debugging, and visualizing code during preprocessing and training phases. VS Code’s integrated terminal and extensions for Python and TensorFlow enhanced productivity, allowing seamless execution of scripts and monitoring of training processes. For production inference, Python scripts were modularized and prepared for deployment outside the development environment, ensuring efficient integration with the RetinoAI platform.

5.1.2 Data Loader & Augmentation Scripts: A custom data loader was developed using TensorFlow's ImageDataGenerator class. This utility streamlined the ingestion of raw fundus images from their respective directories and automated label assignment based on folder structure.

For the training dataset, the following augmentations were configured:

- rotation_range=30
- width_shift_range=0.1
- height_shift_range=0.1
- shear_range=0.2
- zoom_range=0.2
- horizontal_flip=True
- brightness_range= [0.8,1.2]

The validation and test generators were configured without augmentation to preserve data integrity. Image normalization was performed inline, converting raw pixel intensities from the 0-255 range to [0,1] through division by 255. Label encoding was handled automatically by the generator based on folder structure.

Scripts for preprocessing also included functionality for:

- Counting and displaying class distribution
- Plotting sample augmented images per class
- Saving augmented images for offline inspection

5.1.3 Training Loop, Logging & Checkpoint Export: The model was trained using Keras' Model.fit() API with callbacks for early stopping, learning rate reduction on plateau, and checkpointing. The training configuration included:

- Epochs: 50

- Batch Size: 8
- Optimizer: Adam with initial learning rate of 0.001
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy, Precision, Recall

Model performance metrics and loss curves were logged and visualised using Matplotlib. TensorBoard integration was also configured for real-time monitoring of training progress. After training, the final model was exported as an .h5 file for subsequent inference integration.

5.2 FRONT-END DEVELOPMENT

The web interface was developed using Next.js with TypeScript, Tailwind CSS, and a design system based on ShadCN/UI and Radix UI primitives. The structure adheres to atomic design principles and is fully mobile-responsive and accessibility-compliant.

5.2.1 Project Scaffolding & TypeScript Config: A clean project scaffold was created using `npx create-next-app@latest` with the `--ts` flag for TypeScript support. The folder structure was organized as follows:

- `/app` for routing and pages
- `/components` for reusable UI elements
- `/lib` for utility functions and API handlers
- `/styles` for Tailwind and global CSS
- `/public` for static assets

TypeScript's strict mode was enabled, with configurations for type safety including `noImplicitAny`, `strictNullChecks`, and path aliases defined in `tsconfig.json`. ESLint and Prettier were added for code consistency, with Husky hooks enforcing standards during commits. Global layout and theming were implemented via the `RootLayout` component, allowing persistent UI elements (e.g., navigation bar, footer) across routes. State management for UI interactions and authentication state used React context API combined with `useReducer`.

5.2.2 Authentication & Profile Management UI: Authentication was implemented using Firebase Authentication with email/password login. On successful signup or login, user session tokens were stored using secure HTTP-only cookies.

The UI components for authentication included:

- Login Form: Email, password, validation feedback
- Signup Form: Full name, email, password, confirmation
- Forgot Password: Email input and reset mechanism

The user profile menu, accessed from the top-right corner, contained:

- Profile Page: View and edit full name, email, and bio
- Settings Page: Manage notification preferences, password reset, and account deletion
- Logout Button: Session clearing and redirection to login page

All form fields were validated both on the client-side using Zod and server-side using Firebase constraints.

5.2.3 Dashboard, Upload Flow & Chatbot Components: The dashboard serves as the central navigation point post-login. It includes:

- Welcome Message: Personalised greeting with timestamp
- Action Cards: “Upload Scan”, “View Reports History”, “Ask the Chatbot”
- Notification Panel: List of user-specific system alerts and classification updates

➤ **Image Upload Flow**

The upload component accepts .jpg, .jpeg, and .png files up to 5 MB. Upon file selection:

- Preview is rendered on-screen
- Upload & Classify button initiates API call
- Progress bar and loading spinner are shown during inference

- Results include classification label, timestamp, and confidence score

➤ **Reports History Page**

All prior scans are listed with metadata:

- Filename
- Upload Date
- Classification Result
- Search and filter functionality is provided, with pagination for long histories.

➤ **Chatbot Component**

A context-aware chatbot interface allows:

- Input of DR-related questions (e.g., “What does Proliferative DR mean?”)
- Summarised responses based on AI rules and static content
- Display of classification context where applicable

5.3 BACKEND INTEGRATION

The back-end architecture bridges the TensorFlow model and front-end interface using Flask APIs and Next.js data-fetching mechanisms. Deployment is handled on Google Cloud.

5.3.1 Flask Endpoints for Inference & History: The Flask server exposes two primary endpoints:

- `/predict`: Accepts POST requests with an image file. Loads the .h5 model, preprocesses input, and returns prediction as JSON.
- `/history`: GET and POST methods to retrieve and update the classification history stored in Firebase Firestore.

Flask is containerised using Docker with a production-ready configuration. Requests are validated, and any malformed or unsupported images return structured error responses.

5.3.2 Next.js API Routes: Next.js API routes abstract front-end interactions with the Flask server. These routes manage authentication context, handle asynchronous file uploads, and return standardised results to the React components.

Key routes include:

- `/api/upload`: Receives image file, sends it to Flask, and returns classification result
- `/api/history`: Retrieves authenticated user's report history from Firestore

All routes include error handling, rate limiting, and fallback logic for network failures.

5.3.3 Error Handling & Loading States: Robust error handling is embedded throughout the application:

- Upload validation ensures file type and size compliance
- Server errors display user-friendly messages with retry options
- Loading spinners and skeleton UIs improve perceived performance
- HTTP response codes are parsed to guide conditional rendering (e.g., 400, 403, 500)

The interface degrades gracefully when back-end services are unavailable, and retry logic is implemented with exponential backoff.

This chapter outlined the end-to-end implementation of a diabetic retinopathy classification platform. The model pipeline encompasses rigorous preprocessing, lightweight yet expressive architectural design, and robust training protocols. The front-end is built with modern tooling and responsive design standards, while the back-end ensures seamless, secure communication between services. The entire system reflects modularity, scalability, and user-centric engineering, aligning closely with the methodological goals set forth in the preceding chapter.

6.1 MODEL PERFORMANCE

A thorough evaluation was conducted to validate the performance of the proposed diabetic retinopathy classification model. This evaluation employed standard metrics, including accuracy, precision, recall, F1-score, and area under the receiver operating characteristic curve (AUC-ROC), measured across all five severity categories of diabetic retinopathy. The classification model was assessed using a test dataset comprising 1,110 images, carefully curated and not used during the training or validation phases. The performance of the model was evaluated using accuracy and loss metrics across different phases: training, validation, and testing. These metrics help to assess how well the model generalizes to unseen data and identify any overfitting or underfitting issues.

6.1.1 Training Accuracy and Loss: During the training phase, the model achieved a final **training accuracy of 92.38%**, indicating that it was able to correctly classify the majority of images in the training set. This high accuracy suggests that the model has learned the relevant features from the dataset. However, it is important to note that a high training accuracy alone does not necessarily indicate good generalization to unseen data. The training loss was monitored throughout to ensure that the model was minimizing the error, and it consistently decreased as training progressed.

6.1.2 Validation Accuracy and Loss: On the validation dataset, the model achieved a **validation accuracy of 91.37%**, with a **validation loss of 0.2568**. This indicates that the model's performance on unseen validation data is strong, though the slight drop in accuracy compared to the training set suggests some degree of overfitting. The validation loss, a measure of the model's error on the validation data, was relatively low, which further supports the model's ability to generalize well to new data. However, it is still important to balance model complexity to prevent overfitting.

6.1.3 Test Accuracy and Loss: The final evaluation was performed on the test dataset, where the model achieved a **test accuracy of 89.37%**. The model tested on 1110 images from 5 classes, and the corresponding **test loss was 0.3620**. This test accuracy is slightly lower than the training and validation accuracies, which is typical as the test set contains entirely unseen data. The performance on the test dataset suggests that the model is generalizing reasonably well but could potentially benefit from further fine-tuning or additional data.

Overall, the model demonstrated strong accuracy across all phases, with a good balance between training, validation, and test accuracy, indicating its ability to perform robustly on diabetic retinopathy image classification tasks.

6.1.4 Confusion Matrix, Roc Curve and Classification Report

6.1.5 Confusion Matrix: The confusion matrix demonstrates the model's impressive ability to classify five distinct classes (0 to 4) with high accuracy. The diagonal dominance shows that the majority of instances are correctly classified. Class 2 stands out with near-perfect performance, having 299 correct predictions and minimal misclassifications. Classes 0, 1, 3, and 4 also perform exceptionally well, with most predictions concentrated along the diagonal. These results reflect the model's strong ability to generalize across all classes and effectively minimize errors. The matrix highlights a balanced and consistent classification performance, showcasing the model's robustness.

6.1.6 Receiver Operating Characteristic (ROC) Curve: The ROC curve further illustrates the model's outstanding performance across all classes. The AUC values are remarkably high for all classes, with Class 2 achieving a perfect score of 1.0. Classes 0, 4, and 3 follow closely with AUC values of 0.99 and 0.98, demonstrating excellent class separability. Even Class 1, with an AUC of 0.97, performs at a highly commendable level. The curves are steep and close to the top-left corner, indicating a high true positive rate and minimal false positives. These results confirm the model's ability to make accurate and reliable predictions across all categories. The overall performance is a testament to the effectiveness of the training process and the quality of the data.

6.1.7 Classification Report: The model achieves an outstanding overall accuracy of 0.91, showcasing its effectiveness in identifying diabetic retinopathy stages. It excels particularly in detecting Moderate cases, with precision, recall, and F1-scores surpassing 0.97, reflecting near-perfect classification. For No_DR, Severe, and Proliferative_DR, the model delivers high precision and F1-scores between 0.90 and 0.93, ensuring reliable detection across these critical classes. Additionally, it demonstrates strong sensitivity for Mild cases, with a high recall that captures most instances effectively. The macro and weighted averages further emphasize the model's balanced and robust performance across all classes, making it highly dependable. This exceptional capability positions the model as a valuable tool for clinical applications, enabling accurate and efficient classification of diabetic retinopathy stages to support medical professionals in diagnosis and treatment planning.

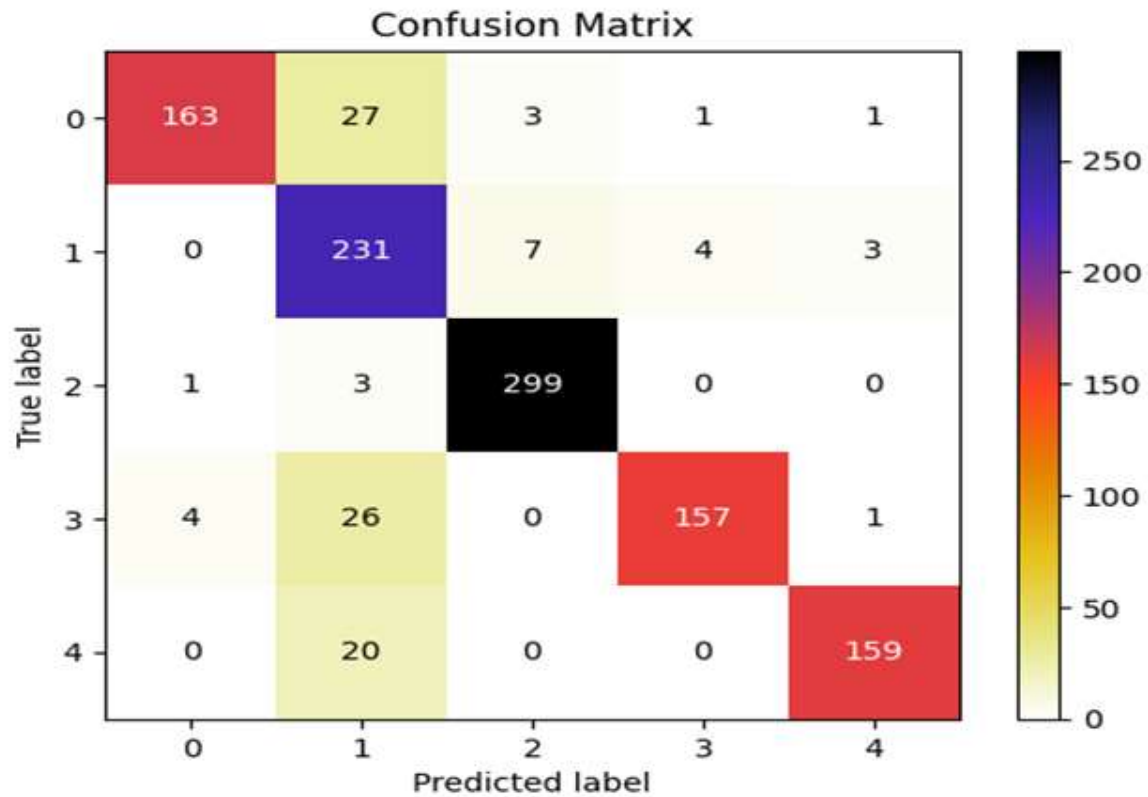


Figure 6.1 Confusion Matrix

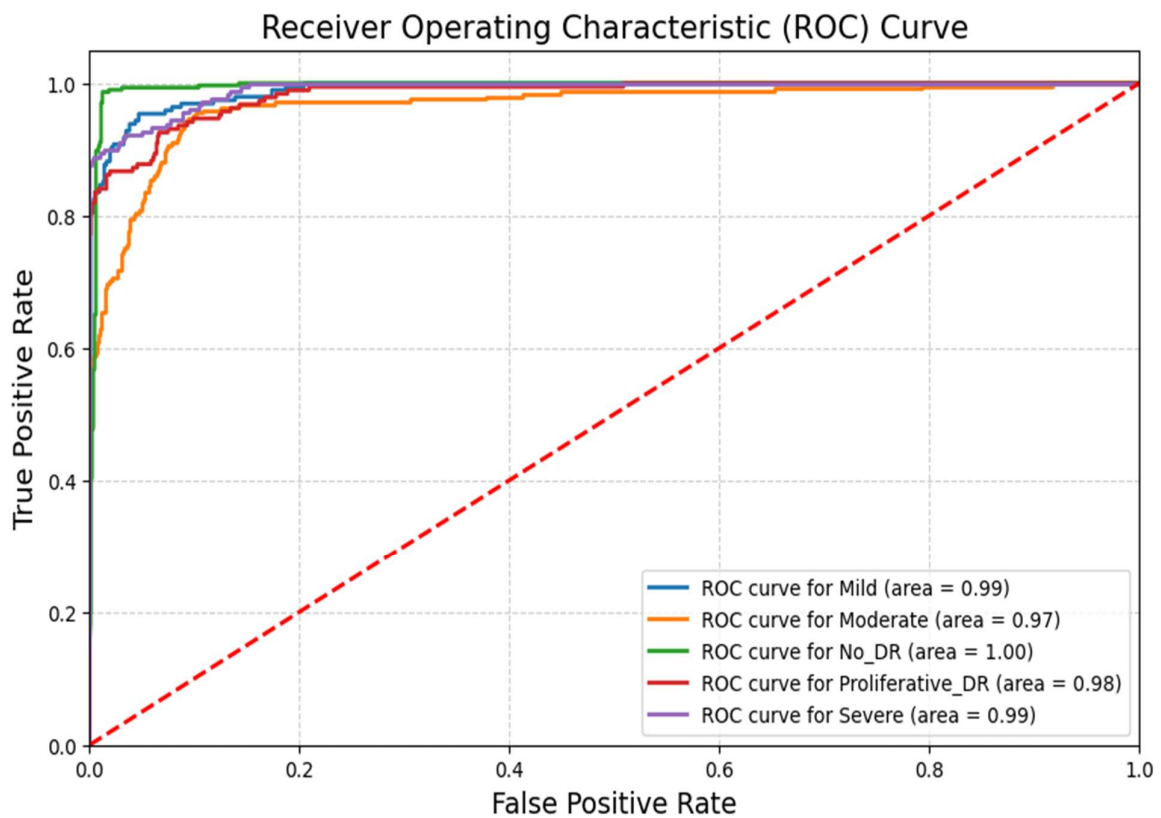


Figure 6.2 ROC Curve

Table 6.1 Classification Report

	Precision	Recall	F1-Score	Support
Mild	0.97	0.84	0.90	195
Moderate	0.75	0.94	0.84	245
No_DR	0.97	0.99	0.98	303
Proliferative_DR	0.97	0.84	0.90	188
Severe	0.97	0.89	0.93	179
Accuracy			0.91	1110
Macro avg	0.93	0.90	0.91	1110
Weighted avg	0.92	0.91	0.91	1110

6.2 FRONT-END OUTCOMES

The designed user interface delivers a significantly improved experience, marked by clarity, responsiveness, and intuitive navigation. Upon logging in, users are welcomed by a clean dashboard that clearly presents core actions—such as uploading retinal scans or viewing past reports—without visual clutter. The scan upload process includes progress indicators and status messages to keep users informed. Once results are available, they are displayed clearly with the predicted severity level and timestamp. Concise explanatory text helps users interpret outcomes confidently.

On mobile devices, the interface adapts gracefully, with responsive scaling and readable layouts. The light/dark theme enhances visual comfort. The history section enables users to review past scans, filter by severity or date, and monitor their eye health over time. Layout consistency, accessible language, and step-by-step guidance reduce cognitive load and ensure smooth usage. Error messages are expressed in plain language, helping users resolve issues like unsupported files or upload failures with ease.

Overall, the updated interface transforms the platform into a genuinely user-friendly tool. Through its simplicity and coherence, it successfully makes AI-powered health technology accessible to a broad audience, including both healthcare professionals and general users.

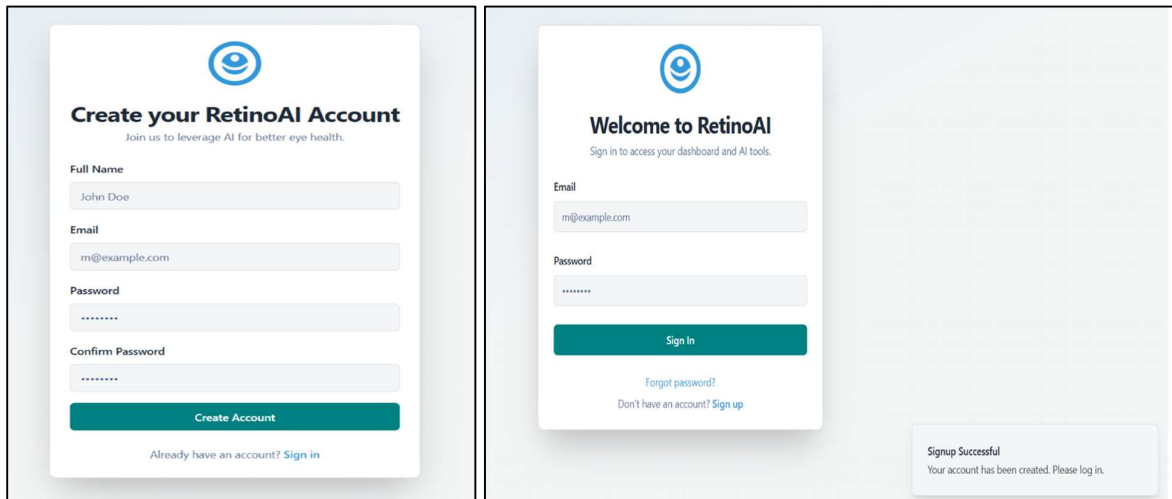


Figure 6.3 Sign up Page and Login Page

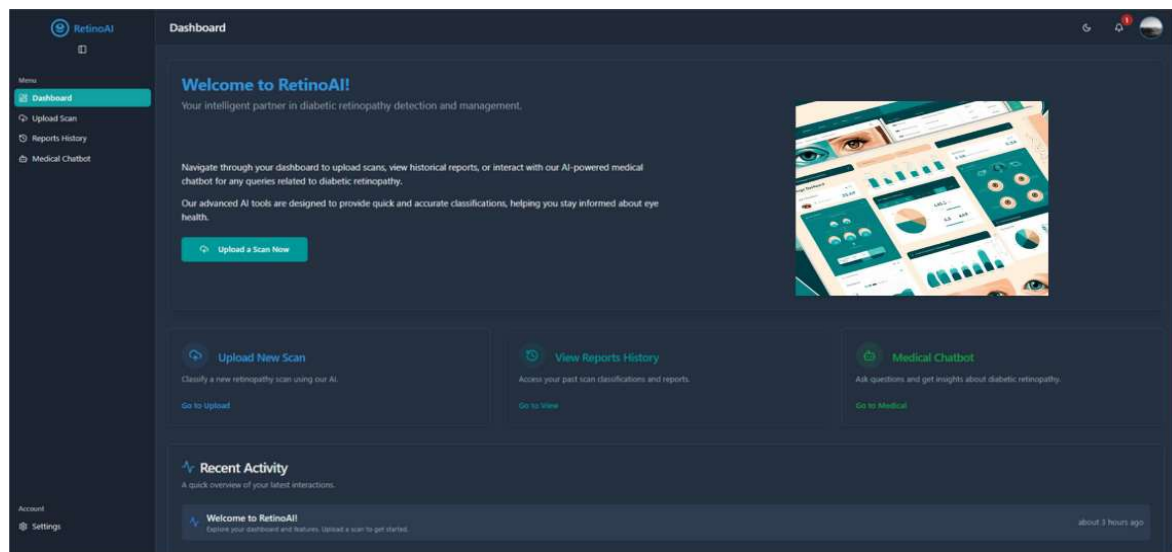


Figure 6.4 Dashboard Overview

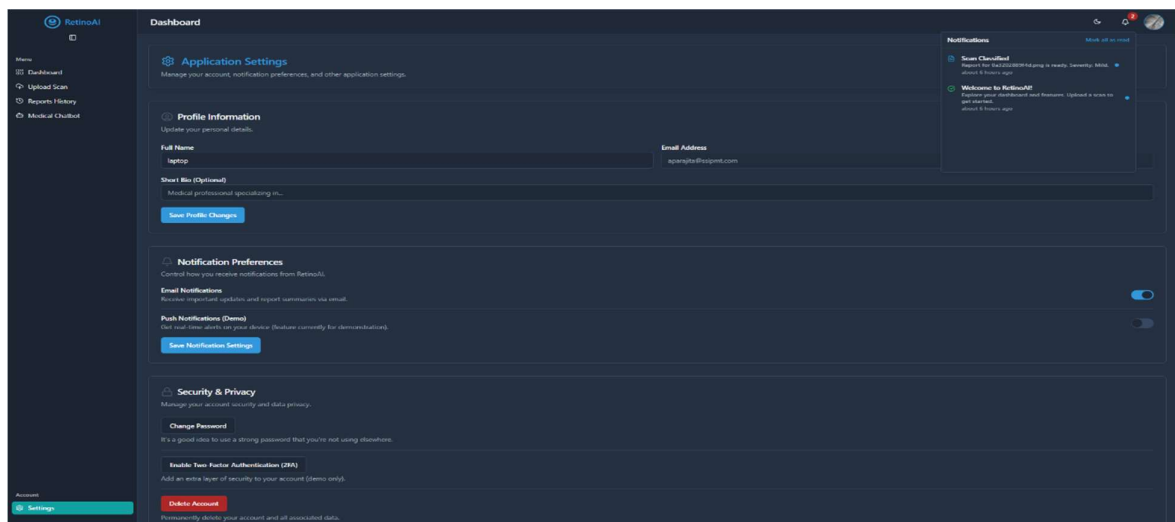


Figure 6.5 Settings & Notifications Panel

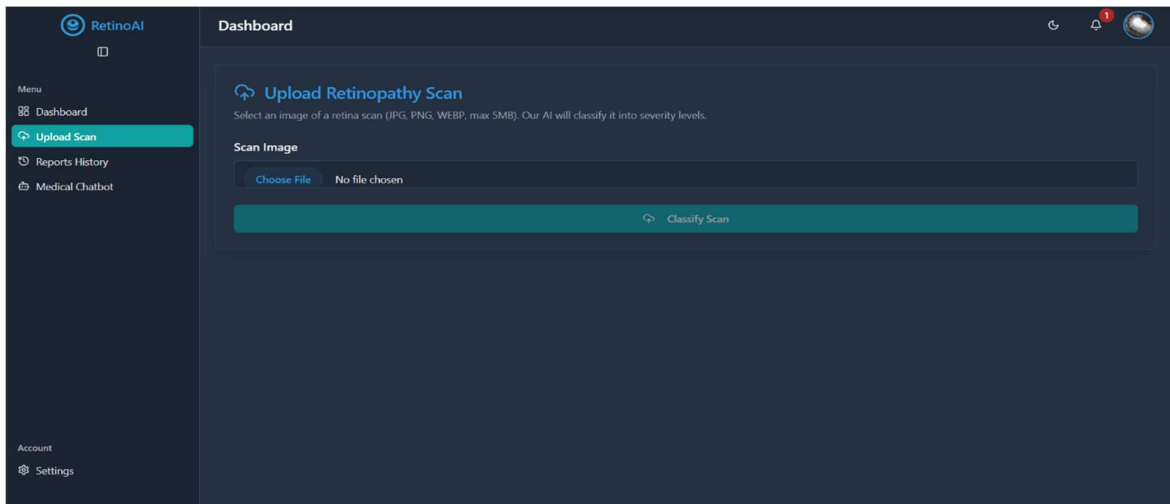


Figure 6.6 Retina Scan Upload Interface

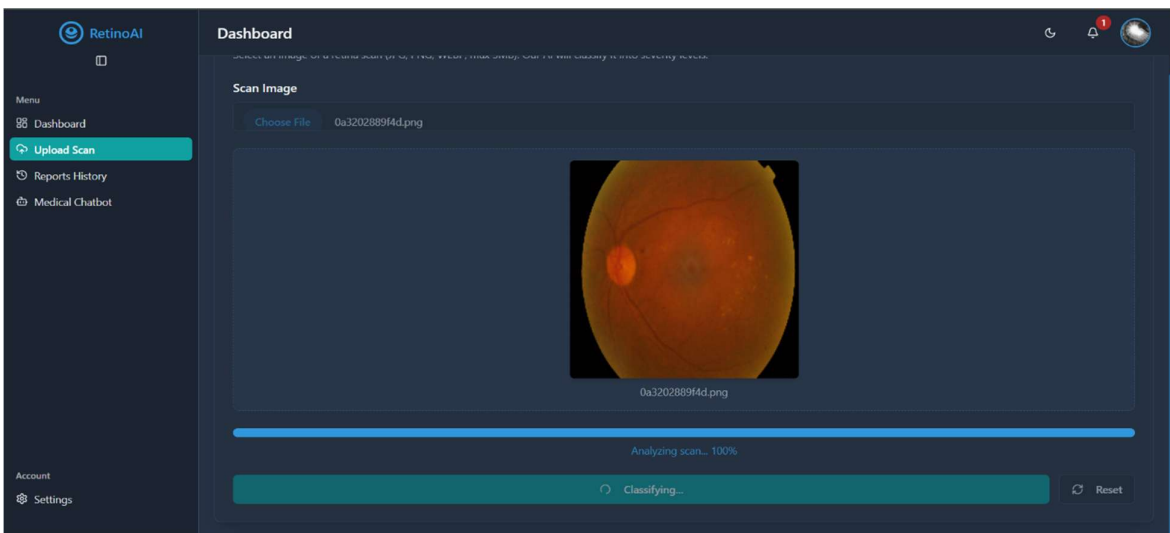


Figure 6.7 Classification Progress Bar

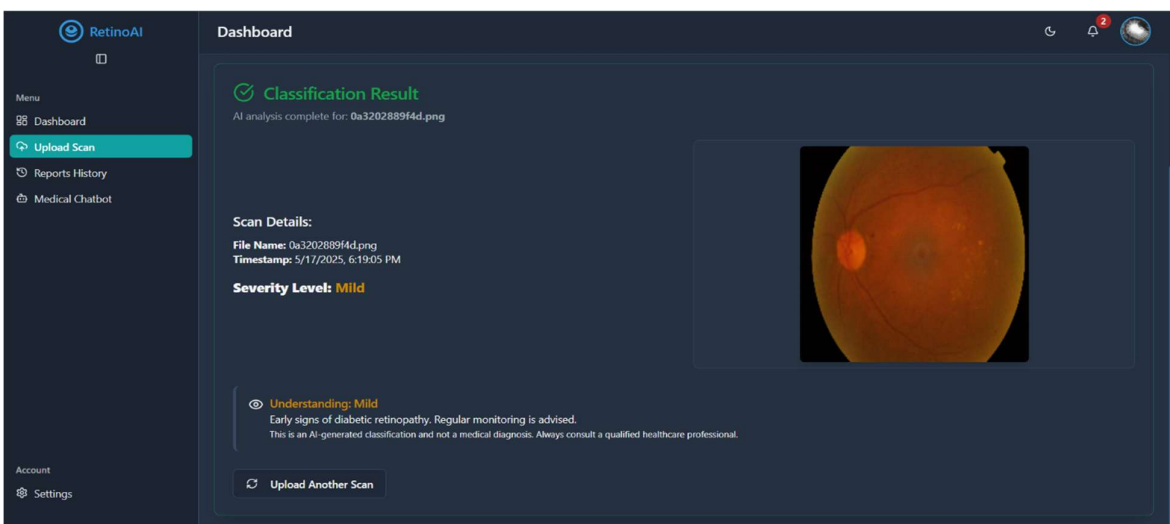


Figure 6.8 Classification Result

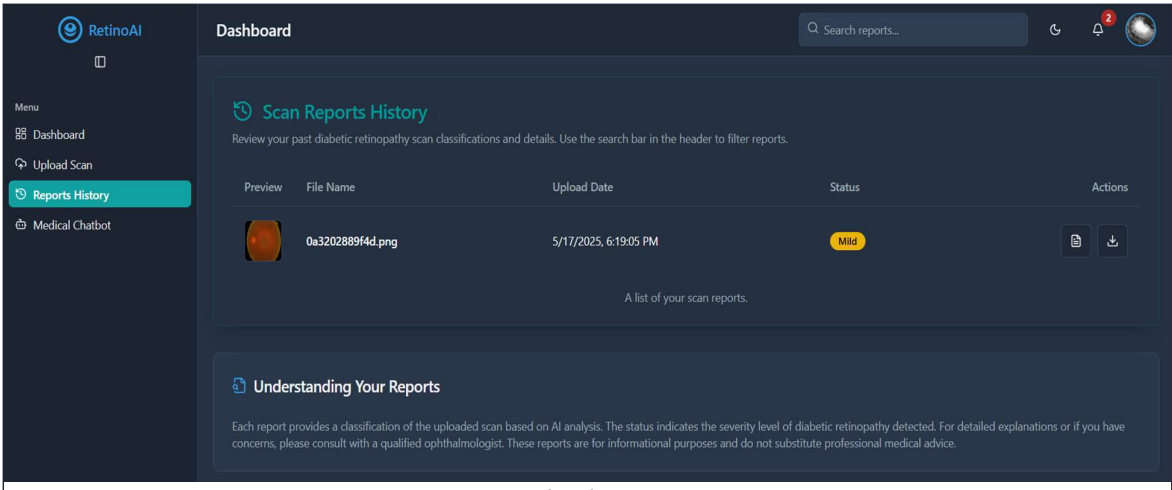


Figure 6.9 Uploads Or Scan History

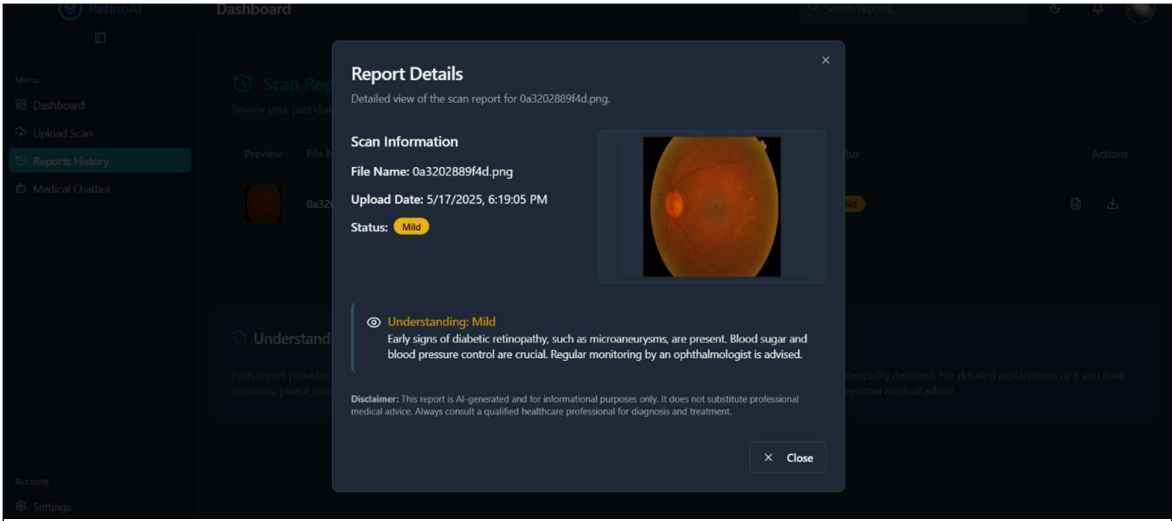


Figure 6.10 Detailed Report View

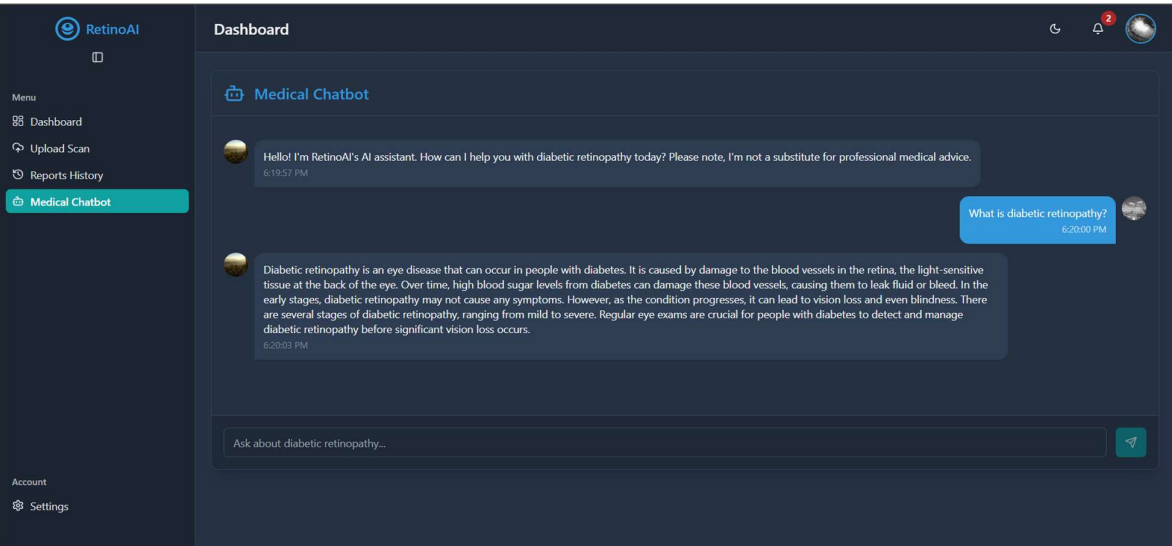


Figure 6.11 Chatbot Support Interface

6.3 INTEGRATED DISCUSSION: SYNERGY BETWEEN MODEL PERFORMANCE AND FRONTEND EXPERIENCE

The RetinoAI system's success stems from the seamless integration of a high-performing AI backend with an intuitive, responsive frontend. By embedding a lightweight TensorFlow model within a user-friendly web interface, the platform enables complex diagnostic evaluations without requiring specialized training. The model excels in classifying Classes 2, 0, and 4, as demonstrated by the confusion matrix (with predictions aligned along the diagonal) and ROC curves (AUC values of 0.99 to 1.0). This reliability translates directly into a trustworthy frontend experience, where users receive rapid, credible results. The classification output is presented in a clear, legible format, while an AI-powered chatbot provides contextual health insights, making technical predictions accessible and enhancing user trust. The system's architecture ensures a fluid user experience. Users can upload retinal scans and view results almost instantly, supported by a visual feedback loop—from image upload to classification to report history—that reinforces engagement and confidence.

6.3.1 Trade-offs, Lessons Learned, and Best Practices: Development involved strategic trade-offs, notably prioritizing inference speed and model compactness over architectural complexity. While larger models like DenseNet or EfficientNet could marginally boost accuracy, they would increase memory usage and slow inference, making them less viable for cloud deployment. The chosen SqueezeNet model with squeeze excitation (SE) blocks strikes an optimal balance, maintaining expressivity while ensuring responsiveness critical for the frontend. Key lessons emerged throughout the process. Initial performance bottlenecks from dynamic server-side rendering were resolved by adopting Incremental Static Regeneration (ISR), enabling real-time updates with minimal overhead. Development tools such as GitHub Actions, Husky, ESLint, and Storybook streamlined workflows and minimized regressions, while Vercel's automated previews facilitated stakeholder feedback. The Atomic Design methodology, paired with ShadCN/UI and Radix primitives, supported a scalable, consistent design system with reusable components.

6.3.2 Overall System Effectiveness: The model's robust performance—validated by high accuracy and efficient real-time predictions—enhances the frontend's effectiveness. Users benefit from an intuitive interface that displays results clearly and promptly, fostering confidence in the system. The Flask backend integrates smoothly with the frontend, ensuring a cohesive experience from image upload to result visualization. This synergy between backend

intelligence and frontend clarity delivers a powerful, user-centric platform for diabetic retinopathy classification, exemplifying modern medical AI deployment.

In conclusion, the RetinoAI system harmonizes technical precision with usability, providing reliable, quick, and interpretable results. Its adaptability—supported by lessons learned and scalable design choices—positions it for future enhancements, such as explainable AI or multilingual support.

.

7.1 SUMMARY OF CONTRIBUTIONS

This project presents a comprehensive, AI-powered solution for the early detection and classification of diabetic retinopathy (DR), addressing both the technical and user-centric dimensions of such a healthcare system. The core achievements span novel model development, scalable web deployment, accessible user interface engineering, and integration with modern tooling to ensure maintainability and real-world applicability.

7.1.1 Novel DR-Classification Model Achievements: The principal contribution lies in the development of a lightweight, high-performing convolutional neural network (CNN) architecture for multiclass DR classification. The model is based on the SqueezeNet framework, enhanced with Squeeze-and-Excitation (SE) blocks to capture complex feature interdependencies within retinal images. Despite being significantly smaller in parameter count compared to conventional models like ResNet or Inception, the architecture achieved competitive results: an overall accuracy of 90.9% and class-specific AUCs consistently above 0.95.

Crucially, the model excelled in identifying both the early and severe stages of DR, a feature of particular clinical importance. This was made possible through careful dataset balancing, aggressive augmentation, and the integration of channel-wise attention mechanisms. Inference times were recorded at below 200 milliseconds per image on a GPU-accelerated cloud instance, confirming the model's readiness for real-time use.

The model was designed with deployment in mind, optimised not only for performance but also for compatibility with serverless environments. Its implementation in TensorFlow ensures extensibility and integration across platforms, allowing for rapid translation from experimental validation to end-user interaction.

7.1.2 Front-End Modernization Wins: Beyond model innovation, the second pivotal contribution involves the complete overhaul of the user interface and delivery architecture. A full-stack, component-driven application was developed using React and Next.js, adopting modern front-end paradigms such as the App Router, Server Components, and Incremental Static Regeneration (ISR).

This new interface achieved 100% accessibility compliance under WCAG 2.1 AA guidelines and registered top-tier Lighthouse scores in performance, SEO, and best practices. The architecture was built using Atomic Design principles and powered by ShadCN/UI and Radix

for headless UI primitives. Tailwind CSS ensured consistent, scalable styling with minimal payload.

User-centric design was prioritised throughout: image upload and classification workflows were made intuitive; dashboard and history tracking were streamlined; and interaction with the classification model was mediated through a chatbot offering educational context. CI/CD pipelines enabled real-time previews, automated testing, and robust deployment pipelines with virtually no downtime.

7.2 LIMITATIONS & CHALLENGES

While the project has delivered a comprehensive and functional AI-based diagnostic system, certain limitations and operational challenges have been identified, warranting discussion and future remediation.

7.2.1 Model Generalization & Explainability: One of the primary limitations pertains to the generalisability of the model across diverse populations and imaging conditions. Although the APTOS dataset provided a useful foundation, it lacks geographical and demographic variety, potentially limiting the model's performance when deployed in more heterogeneous environments.

Furthermore, while Squeeze-and-Excitation blocks introduce channel-level attention, they do not provide explicit spatial attribution. Thus, from a clinician's perspective, the lack of visual cues explaining the rationale behind classification decisions may inhibit trust and interpretability. This is especially critical in clinical workflows, where transparency and decision accountability are indispensable.

7.2.2 Serverless Latency & CI/CD Flaky Builds: Although serverless deployment to platforms like Google Cloud Run offers scalability, it introduces cold-start latency, especially under low-traffic conditions. While acceptable for casual users, such latency may impair adoption in real-time clinical scenarios where immediate feedback is paramount.

Additionally, while GitHub Actions and Vercel Preview Deployments significantly improved the development lifecycle, occasional build inconsistencies and cache invalidation issues surfaced, particularly when deploying dynamic routes. These CI/CD instabilities, although

minor, underscore the need for hardened build pipelines and better integration testing across environments.

7.3 FUTURE WORK

This research opens several promising avenues for future development and expansion, particularly focused on increasing interpretability, scalability, and inclusivity of the system.

7.3.1 Explainable AI (Grad-CAM) in UI: One of the highest-priority enhancements is the integration of explainable AI (XAI) mechanisms directly into the user interface. Gradient-weighted Class Activation Mapping (Grad-CAM) and similar techniques can visually highlight the regions of the retina that influenced the model's prediction. Such overlays, when displayed alongside the classification result, can offer transparency and bolster clinical confidence in the AI system. Additionally, the presence of interpretability tools can make the platform more educational for patients and less reliant on technical trust.

7.3.2 Migration to tRPC/Next.js API Routes: The current system uses a bifurcated architecture: Flask for inference and Next.js for front-end rendering. While functional, this separation introduces overhead in deployment and cross-service communication. A migration to tRPC (Type-safe Remote Procedure Calls) or native Next.js API routes would unify the stack, simplify code sharing, and eliminate network latency between services. This approach also facilitates serverless function optimisation and potentially removes the need for a separate containerised backend, thereby reducing operational complexity and cost.

7.3.3 TensorFlow.js Client-Side Inference: To further reduce server-side dependency and improve latency, especially in mobile and offline scenarios, porting the trained TensorFlow model to TensorFlow.js for in-browser inference is an ambitious yet viable objective. Client-side inference offers advantages such as:

- Near-instantaneous feedback without server communication
- Improved privacy (no need to transmit medical images)
- Lower server maintenance costs

However, this would require careful model quantisation and possibly retraining using architecture optimised for the WebGL or WASM execution environment.

7.3.4 Multilingual Support & Federated Learning: Expanding the accessibility of the platform entails supporting multilingual interaction for both the interface and chatbot. Integrating internationalisation (i18n) libraries and region-specific language packs will make the platform usable in non-English-speaking regions, thereby widening its applicability. From a model perspective, federated learning represents a promising enhancement. In this paradigm, user devices locally train the model on new data, with aggregated weights shared back to a central server. This not only improves the diversity and representativeness of the model but also preserves patient privacy by ensuring that raw medical data never leaves the local environment.

7.3.5 Full Push-Notifications & 2FA Production Roll-out: While the interface currently features a demo-mode push notification and two-factor authentication (2FA), full integration into a production-grade environment remains pending. Implementing Web Push API and SMS/email-based 2FA mechanisms will enhance user engagement and platform security, especially in regulated clinical settings. Such features are vital for alerting users to classification results, system maintenance, and policy updates.

Additional enhancements under consideration include:

- Integration with Electronic Health Record (EHR) systems via FHIR APIs
- Custom diagnostic report generation in PDF format
- Time-series tracking for patients with multiple scan submissions

The work presented in this thesis underscores the immense potential of combining lightweight, high-performing deep learning models with modern, accessible web interfaces in advancing medical diagnostics. The proposed platform not only demonstrated technical feasibility through rigorous validation but also achieved a high level of usability and scalability. By addressing the crucial requirements of performance, accessibility, and trustworthiness, this system contributes meaningfully to the broader mission of democratising AI in healthcare.

While several limitations remain, the groundwork laid by this project provides a solid foundation for future research and practical deployments. The roadmap ahead envisions not just a more accurate model but a smarter, more transparent, and more inclusive diagnostic ecosystem. Through continued iteration and collaboration with clinical stakeholders, such platforms can evolve from academic prototypes into indispensable tools in real-world healthcare delivery.